

Two UAVs, two demonstrations

A step-by-step ROS 2 / Gazebo kinematics guide

Prepared for Ismaël Assoumane Idi

20 September 2026

Purpose. Learn to implement deterministic ascent, travel, yielding, spherical-obstacle bypass and landing. These are kinematic demonstrations with explicit geometric clearance checks. They do not model aerodynamic forces, rotor thrust, tracking error or physical flight control.

Research context: *Constraint Consensus-Based Dynamics for Decentralized Path Planning in UAV Swarms*. This showcase does not implement TCC, Sun-Sun, consensus or an optimization solver. Its geometry and paths are separate from the numerical campaign.

Contents

1	What you will build	2
2	Software and prerequisites	2
3	Enter, build and inspect the workspace	3
4	Run and record each wave	3
5	Coordinates, dimensions and the scenario	4
6	Code lesson 1: vectors and movement	5
7	Code lesson 2: deterministic mission states	5
8	Code lesson 3: check between updates	6
9	Code lesson 4: connect the model to Gazebo	6
10	Code lesson 5: the live clearance display	7
11	Verification and actual results	8
12	Visual Studio Code and troubleshooting	9
13	Rebuild this guide and recover work	10
	Official references	10
14	Where to edit each part	10
A	Complete first-party source listings	12

1 What you will build

Wave 1 starts with two parked UAVs. They ascend to model-origin height 2.8 m, cross toward opposite destination pads and descend. Both take off simultaneously. When a one-second prediction detects an approaching conflict, UAV 2 holds in flight and gives UAV 1 priority. It resumes at the first update where the remaining crossing legs have sufficient predicted clearance. Wave 2 adds a stationary red sphere. Each UAV follows a bypass on its own side and then crosses to the opposite destination pad using the same priority rule.

The four mission stages are READY, TAKEOFF, TRAVEL/BYPASS and LANDING. YIELD is a deliberate wait in both waves. LANDED denotes arrival; COMPLETE requires both vehicles to have arrived. FAILED denotes a detected problem. A timeout is not successful obstacle avoidance.



Figure 1: Actual Gazebo Wave-1 rendering during the pause/resume verification. The orange vehicle yields while the blue vehicle proceeds. The live counter shows measured separation.

The original paused environment remains available through its original launcher. Each demonstration has its own package and uses a separate world named `um6p_demo`. Its motion plugin is absent from the paused world. Only the new demonstration advances simulation time after an explicit start command.

2 Software and prerequisites

ROS 2 runs the launch process and receives telemetry. Gazebo owns the scene, simulation clock and model positions. `ros_gz_bridge` translates Gazebo state messages and clock messages into ROS messages. A workspace contains packages; building a package installs its files so launch can resolve them by package name. An SDF file describes the world. A C++ system plugin runs inside Gazebo's update loop.

The tested native stack is Ubuntu 24.04.5 amd64, ROS 2 Jazzy, Gazebo Harmonic Sim 8.15.0 and `ros_gz` 1.0.24, with Intel Iris Xe/Mesa hardware rendering. The installation already existed; no driver or system upgrade was performed. This is the same supported pairing as the campus

preparation. Earlier user-reported patch versions, a freshly provisioned OS, Docker and software-only rendering are not claimed tested.

Prerequisites: the complete updated delivery folder, a Bash terminal, the existing ROS/Gazebo installation, a C++ compiler and colcon. GUI commands additionally need a working graphical desktop. The original native dependency list and the added demo dependency list record package requirements; observed versions are evidence, not a permanent apt lock. All project assets are local after setup. No runtime model download is required.

3 Enter, build and inspect the workspace

Commands in this guide are literal Bash commands except example paths, which you must replace. Do not type a shell prompt character. Quote paths containing spaces. Unless stated otherwise, work from the delivery root.

Working directory: any terminal initially. Purpose: enter the extracted folder and check installed prerequisites. Expected result: the check prints OS/tool versions and the plan reports missing dependencies or confirms they exist. These checks do not install software.

```
cd "/your/path/ros2_gazebo_um6p_campus"  
bash scripts/check_environment.sh  
bash scripts/setup.sh --plan
```

The additive setup script checks both dependency manifests. To install missing packages on a compatible machine, from this same directory run `bash scripts/setup_demo.sh -install-missing` after reviewing its plan. This requires sudo and configured official apt repositories; expected result is the missing packages installed without upgrading existing ones. This installation command was not needed on the tested machine. The new C++ package uses the existing `gz_sim_vendor` and `gz_plugin_vendor` development exports, plus Gazebo GUI and Qt 5 Quick development packages for the counter overlay. See `references/demo-packages.txt`. If they are absent, use the original tutorial's explicit installation procedure after reviewing the proposed apt transaction. Do not replace working ROS installations to follow this guide.

Working directory: delivery root. Prerequisite: dependencies present. Purpose: compile and install all five packages locally. Expected result: colcon reports five packages finished, including `um6p_demo_core` and both scenario packages.

```
bash scripts/build.sh
```

Build and install products remain inside this workspace. Runtime uses copied installed assets and the installed shared library. Source changes require a rebuild. No personal shell configuration is required: the launch wrappers source ROS and this workspace explicitly.

4 Run and record each wave

Working directory: delivery root in Terminal A. Prerequisites: build succeeded and no other campus demonstration is running. Purpose: open Wave 1. Expected result: Gazebo opens with both UAVs parked and simulation time paused at zero. This command stays attached to Terminal A.

```
bash scripts/launch_demo.sh 1 gui
```

Working directory: the same delivery root in Terminal B. Prerequisite: the scene has loaded. Purpose: start the demonstration clock and deterministic motion. Expected result: the service replies `data: true`; after three seconds of simulation time, ascent begins.

```
bash scripts/demo_control.sh start
```

The start command is intentional. This phase was separately authorized to advance kinematic simulation time. The motion plugin reads simulation time, never wall-clock time, for motion. Pausing freezes both positions and mission progress. The following commands, each from Terminal B while Terminal A owns the scene, pause and resume it. Expected result: **data: true** each time.

```
bash scripts/demo_control.sh pause
bash scripts/demo_control.sh resume
```

For a recording, first load the paused scene, position the camera using mouse navigation, start your desktop screen recorder, then issue the start command. Record until both UAVs land. You may pause to explain a view. The researcher supplied two original MP4 recordings, now preserved unchanged in the respective demo media/videos folders. The recording software and its exact capture settings were not independently verified. The screenshot figures in this guide came from Gazebo itself.

Shutdown: press **Ctrl+C once in Terminal A** and wait for clean child exit messages. To repeat from the initial state, close and relaunch. Restarting the process is the supported deterministic reset; in-place Gazebo world-reset behavior is not implemented or claimed verified.

After Wave 1 has stopped, Terminal A can open Wave 2 with the following command. Terminal B then uses the same start/pause/resume commands. Expected result: a large stationary red sphere appears and the two vehicles bypass it on opposite sides.

```
bash scripts/launch_demo.sh 2 gui
```

Wait for the sphere and both UAVs to appear. In Terminal B, from the same delivery root, enter this separate command:

```
bash scripts/demo_control.sh start
```

The expected reply is **data: true**. Do not paste the two terminal commands onto one line. The start request does not launch Gazebo. A successful build alone does not launch Gazebo either.

Keep one demonstration running at a time. The wrappers use ROS domain 85 and Gazebo partition **campus_demo**; these differ from the paused-environment defaults. If deliberately overriding the partition, use the same value in both terminals. Headless operation uses **headless** in place of **gui**; it advances the same kinematics after start but opens no window.

5 Coordinates, dimensions and the scenario

Coordinates are right handed: +X across the courtyard, +Y toward the buildings, +Z upward. Length is in metres, time in seconds, angle in radians. No geographic registration is claimed. An SDF pose is x, y, z , roll, pitch, yaw.

Quantity	UAV 1	UAV 2
Start model origin	(−3.6, −8.2, 0.12)	(3.6, −8.2, 0.12)
Fixed yaw	+0.3	−0.3
Wave 1 destination	(3.6, −40, 0.12)	(−3.6, −40, 0.12)
Wave 2 destination	(3.6, −40, 0.12)	(−3.6, −40, 0.12)
Cruise model-origin height	2.8	2.8

The sphere has fixed center (0, −22, 2.4) and radius 2.4 m; it touches the courtyard plane. The nominal Wave-2 waypoint (0, −22, 2.8) would lie inside the inflated obstacle. The fixed-side heuristic replaces that blocked route with waypoints (±4.3, −19, 2.8), (±4.3, −27, 2.8) and the opposite-side destination at cruise height, followed by descent. These are newly designed showcase routes, not a replay of an archived numerical plan.

The model-origin corridor is $x \in [-7, 7]$, $y \in [-41, -7]$, $z \in [0.12, 2.8]$. The speed limit is 0.85 m/s, and the simulation step is 0.02 s. Yaw is fixed, and rotors are not animated. Velocity may change direction at waypoints; acceleration, jerk and actuator feasibility are not modeled.

The clearance sphere of each UAV is centered 0.36 m above its model origin. Inspection of all mesh vertices gives maximum radius approximately 1.136261 m, so the demonstration uses a conservative 1.2 m bound. This bound includes the propeller bars and landing skids. Its overlap with the ground while parked is a conservative-volume artifact; ground/pad clearance is checked separately using actual mesh height. Minimum local mesh height is 0.005 m, and the pad top is 0.12 m.

6 Code lesson 1: vectors and movement

Start with `include/Motion.hh`. `Vec` provides addition, subtraction, scalar multiplication, dot product and Euclidean length. These are the only operations needed for the position-level model.

To move toward a waypoint g , form $d = g - p$. If $\|d\|$ is at most $v\Delta t$, return g exactly. Otherwise return

$$p_{\text{proposal}} = p + v\Delta t \frac{d}{\|d\|}.$$

This is implemented by `toward`. It limits travel per update and avoids overshooting a waypoint. Reaching a waypoint advances a route index. No solver is called.

Exercise: on paper, evaluate one update from $(0, 0, 0)$ toward $(1, 0, 0)$ with $v = 0.85$ and $\Delta t = 0.02$. The expected displacement is 0.017 m. Then consider a waypoint only 0.005 m away: it must be reached exactly, without a division by zero or oscillation.

7 Code lesson 2: deterministic mission states

Each scenario owns two lines of waypoint triplets in its `config/routes.txt`. The generator embeds them in that scenario's SDF. `Routes.hh` parses these fields, and the `Motion` constructor receives the resulting waypoint sets. Shared motion rules are implemented once in the core package. `phase` explains what each UAV is currently doing. `advance` computes both movement proposals from the same old state and then checks them jointly before assigning the new positions.

Both waves predict the next second of joint travel toward the current waypoints. If the closest predicted pair distance falls below 3.50 m, UAV 2 holds. Once holding, it evaluates the entire remaining current legs at constant speed, including the interval after the first arrival. The two arrival times divide this prediction into linear intervals. It resumes immediately when every such interval stays at least 3.52 m apart. This small release cushion prevents repeated stop/start switching near the threshold. The independent execution guard still enforces the separate 3.20 m requirement on every accepted update.

This is a fixed priority heuristic using exact scenario positions, not onboard sensing or distributed negotiation. There is one continuous yield episode per wave in the final tests. It does not wait for an actual clearance violation. The word “immediate” refers to the first safe release update under this prediction rule, not merely a momentary distance above the minimum.

Wave 2 selects its bypass side from vehicle identity, then joins the crossing route beyond the sphere. The tighter lateral offset of 4.3 m brings it close to the obstacle while maintaining the conservative bound. It is a fixed-scenario heuristic, not a planner for arbitrary obstacles. Changing geometry, routes or vehicle count requires revalidation. Exactly two vehicles are supported here.

If the proposed joint step fails a clearance or corridor check, both UAVs hold their last state. If completion has not occurred by 120 s of mission time, the program reports FAILED. A hold is

safe under this position-level model when the previous joint state was valid; it is not a physically realizable emergency stop guarantee.

8 Code lesson 3: check between updates

Two separated endpoints can conceal a collision between them. For example, relative positions $(-2, 0, 0)$ and $(2, 0, 0)$ both have length 2, yet their connecting segment passes through zero.

For relative segment endpoints a, b , define $d = b - a$. If $d \neq 0$, compute

$$s = \max \left(0, \min \left(1, -\frac{a^\top d}{d^\top d} \right) \right), \quad D = \|a + sd\|.$$

If $d = 0$, use $D = \|a\|$. `segmentDistance` implements this closest-point formula. For two UAVs, $a = p_1^k - p_2^k$ and $b = p_1^{k+1} - p_2^{k+1}$. Both use the same interval parameter. For a fixed sphere, subtract its center from each endpoint of the UAV bounding-center segment.

Required distances are

$$D_{12} \geq 1.2 + 1.2 + 0.8 = 3.2 \text{ m}, \quad D_{i,o} \geq 2.4 + 1.2 + 0.6 = 4.2 \text{ m}.$$

The implementation also requires a small 10^{-6} m numerical cushion. These are engineering demonstration parameters, not modifications of the research benchmark. The formula follows the geometry of Decisions 03 and 04 under synchronized piecewise-linear motion and a static sphere. Binary floating-point checks with large observed margins are not the separate exact-rational U4 certificate of the numerical campaign.

The independent Python verifier recomputes these minima from recorded Gazebo entity-component positions at every update. The scenery checker bounds mesh triangles by axis-aligned boxes and checks the fixed flight region, treating ground and destination pads separately. This is deliberately conservative. It does not claim that arbitrary decorative shapes are mathematically equivalent to spheres.

9 Code lesson 4: connect the model to Gazebo

`CampusKinematics.cc` is a small C++ system plugin. `Configure` reads the wave number, prepares a telemetry publisher and optionally opens a trace file. `PreUpdate` locates the two model entities, returns immediately while paused, validates the 0.02 s step, checks for unexpected external pose changes and applies an accepted joint movement. `PostUpdate` reads the entity poses back and publishes state.

The plugin writes both model poses in one simulator update thread. It does not send asynchronous independent movement commands to two nodes. The Gazebo dynamic-pose stream carries those changes to the renderer. No Physics system, motor model, sensor system or flight controller is loaded. Simulation time advances because the demonstration clock is running; it need not imply force integration.

The generated worlds reuse installed campus meshes. Only UAV static flags change in those separate worlds; destination pads, a paved extension and the optional sphere are added. The extension is 26 m wide and 20 m long, at ground height, covering $y = -45$ to -25 ; it joins the original courtyard. Existing buildings, pergola, landmark and landscaping are unchanged. The original world is untouched. `demo.launch.py` resolves installed package locations, loads the shared library and starts a one-way telemetry bridge. There is no ROS motion-command topic.

Working directory: delivery root. Prerequisite: stopped demonstration and an intentional edit to the world-generation recipe. Purpose: regenerate its two SDF files and reinstall changes. Expected result: two worlds generated, then a successful build. This is unnecessary for ordinary launching.



Figure 2: Actual Gazebo Wave-2 rendering. Both vehicles pass close to the fixed sphere. The obstacle counters turn orange while the required clearance remains satisfied.

```
python3 common/scripts/generate_demo_worlds.py
bash scripts/build.sh
```

When editing constants, reconcile the motion header, generated world geometry and independent verification thresholds. Their independent checks are intended to catch inconsistency. Do not loosen a test merely to hide an unsafe route.

10 Code lesson 5: the live clearance display

The upper-right panel belongs to Gazebo itself. `ClearancePanel.cc` receives the same state message that is bridged to ROS. Its transport callback queues a Qt GUI-thread update; `ClearancePanel.qml` formats the latest distances. The panel follows the right edge as the window changes size. It shows one counter in Wave 1 and three in Wave 2. The C++ capture service saves the actual full Gazebo window, including the panel.

Each large value is a center-to-center distance. The pair minimum is 3.20 m, which includes both 1.20 m vehicle bounds and 0.80 m required surface gap. Each sphere minimum is 4.20 m, including the 2.40 m obstacle radius, 1.20 m vehicle bound and 0.60 m required gap. The smaller margin is the displayed distance minus that minimum. These definitions belong here in the guide; the screen uses concise labels.

A margin of at least 0.80 m is green. A nonnegative margin below 0.80 m is orange. A negative margin is red. Color is accompanied by CLEAR, NEAR LIMIT or VIOLATION text, so color alone is not required to interpret it. Data older than 1.5 wall-clock seconds becomes gray and shows NO LIVE DATA. A paused scene still publishes stationary telemetry, allowing the panel to remain live without advancing time.

To understand the code, follow one message: the system reads the two executed model positions, adds the body-center offset when computing obstacle distances, serializes the values, and publishes

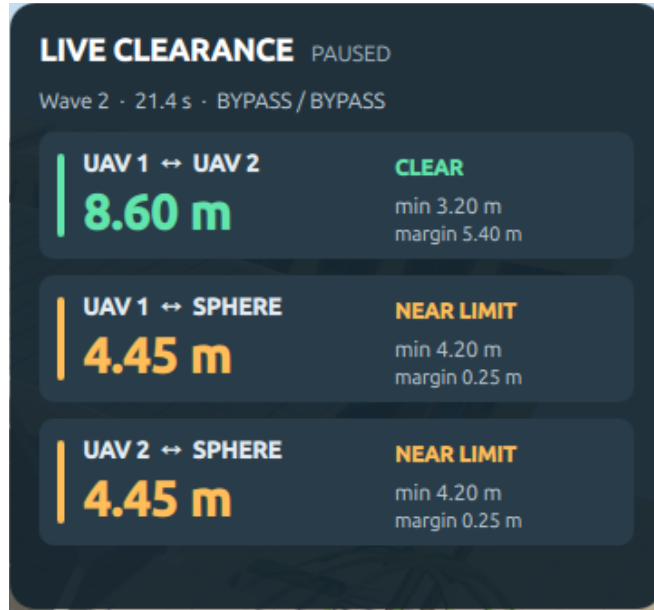


Figure 3: Enlarged detail from the actual screenshot above. Both sphere distances are orange and remain above the 4.20 m minimum. This is a crop of Gazebo evidence, not a synthetic interface.

them. The GUI parses the message, selects its wave-dependent row count, then evaluates each row’s margin. It neither commands UAVs nor changes the acceptance guard. Red behavior is tested using synthetic QML input in an isolated test, without moving a Gazebo vehicle into danger.

Working directory: delivery root after a successful build. Purpose: exercise the motion core and the actual QML threshold functions without launching Gazebo. Expected result: two completed core cases and a JSON PASS with 20 panel cases. The offscreen Qt setting applies only to this widget test, not to a claim of software-rendered Gazebo verification.

```
bash scripts/verify_unit.sh
```

11 Verification and actual results

Working directory: delivery root. Prerequisites: build succeeded and no other demonstration is running. Purpose: run a bounded test with its own isolated Gazebo partition and ROS topic/node namespace. Expected result: a JSON result with PASS, mission completion, measured minimum distances and clean exit. Each wrapper enforces a 150-second wall-time timeout. Output defaults to a new timestamped evidence directory.

```
bash scripts/verify_demo.sh 1 headless
bash scripts/verify_demo.sh 2 gui
```

Run these sequentially. The GUI test saves a real screenshot at a paused midflight point. It verifies initial pause at zero time, starts motion, pauses during flight, checks unchanged positions/time, resumes, checks both landings, compares rendering poses to telemetry, checks ROS clock delivery and stops the launch owner. It then recomputes interval clearances from the full trace. The trace is actual execution evidence; the screenshot alone is not proof of safety.

September 20 cross-talk correction. A user Demo-2 failure log contained a wave=1 state received from a concurrently opened Demo 1. The old verifier shared ROS topics even though it isolated Gazebo. The corrected harness remaps state and clock to a unique ROS namespace and checks scenario identity. A paused-state barrier requires a new sample at or beyond the latest observed tick, then the original exact pose/time equality assertion remains in force. A regression

test runs Demo 2 alongside a separate paused Demo 1. It saves all received states for diagnosis. Manual launch commands remain unchanged.

The automated verifier owns and closes its scene; it is not a presentation launcher. Failure cleanup is expected and is not itself a crash in the manual demo.

Early diagnostics are preserved separately from the final runs. One clean-copy diagnostic could not obtain the scene service response through the Python binding; bounded command-line service exchange resolved this. The first headless diagnostic completed Wave 1 but failed its final check because the scene-description service retained initial poses. The corrected harness uses `dynamic_pose/info`, the renderer’s motion stream. That diagnostic is retained as FAIL rather than silently relabeled. Final results and clean-copy checks are listed in `evidence/VERIFICATION_CURRENT.md`; they govern the tested scope.

Standalone core checks exercise crossing segments, a sphere-crossing segment, deterministic replay and timeout failure. The final GUI tests additionally establish actual rendering, exact entity count, pause/resume behavior, ROS message exchange and clean termination. A new-directory build checks installed-path portability; this does not establish fresh-OS provisioning.

Observed quantity	Wave 1	Wave 2
Completion simulation time (s)	51.92	54.22
Minimum UAV center distance (m)	3.531609	3.530266
Minimum sphere-center distance (m)	not applicable	4.366646
Yield duration (s)	4.26	4.86
Pause/resume and renderer checks	PASS	PASS
Clean exit code	0	0

These are observed GUI results, not illustrative outputs. All interval checks passed. The UAV-pair requirement is 3.2 m and the sphere-center requirement is 4.2 m. Wall-clock duration may differ with rendering load and inspection pauses.

12 Visual Studio Code and troubleshooting

Open the delivery root with File > Open Folder. Use Terminal > New Terminal for Terminal A and create a second terminal for control commands. Use the same commands as above. The included tasks provide build, parked campus, both demos and start/pause shortcuts. The two-terminal commands are the simplest recording workflow. Interactive VS Code UI itself is not a tested runtime route.

No movement: the scene starts paused. Issue start from another terminal in the same partition. Wait three simulation seconds after start. Check that the demo world, not the original paused world, is open.

Control service timeout: confirm the launch is still running and both terminals use the same Gazebo partition. Close a conflicting scene. Do not target the old world or change system security settings.

Library or package missing: rebuild, then launch through the wrapper so both ROS and the local install are sourced. A source directory alone does not make an installed package discoverable. Do not copy an old install directory to a new path.

Blocked route/FAILED: retain the trace and failure result. Inspect waypoint, radius, corridor and sphere consistency. An arbitrary new layout is not guaranteed to work with this two-scenario heuristic.

Unexpected pose modification: another tool changed a UAV entity. Close the scene and restart it. Do not use transform tools or additional controllers alongside this demo.

Blank GUI: use the same desktop graphics session as the working original campus. This task used hardware rendering; software/EGL behavior remains unverified. No graphics-driver replacement is part of recovery.

Safe cleanup: stop Terminal A first. The build, install, log and .runtime directories inside this workspace are disposable. Preserve source, evidence, tutorials and checkpoints. Avoid broad process-kill commands that could affect unrelated work.

13 Rebuild this guide and recover work

Working directory: delivery root. Prerequisites: latexmk or Tectonic on PATH, plus Poppler for inspection; the original guide documents optional TeX dependencies. Purpose: regenerate imported source listings and compile this exact guide. Expected result: `tutorial/02_Two_UAV_Kinematic_Demos_Coding_and_Verification.pdf` and PDF metadata.

```
bash scripts/build_kinematic_tutorial.sh
```

The appendix imports maintained source directly. The generated vertex tables remain in the original campus package and need not be copied into this second guide. TeX dependencies may download at documentation-build time; scene runtime has no such requirement. PDF binary equality across TeX versions is not promised.

On resume, read the current checkpoint, check which source and evidence files exist and compare hashes before rebuilding anything. Preserve intervening user edits. Do not repeat completed experiments just to fill a log. Save progress after meaningful milestones and before expensive operations; a cache or session may not survive a later invocation.

The original paused environment was published first as GitHub [commit 3e8f518](#). The current scientific context is Model 0-D v1.6, with its own three-attempt campaign and exact-oracle limitations. Those outcomes belong to their original evidence. This new showcase neither executes nor certifies TCC/Sun-Sun, and does not replace its benchmark.

Official references

1. [Gazebo Sim 8: creating system plugins](#), update interfaces, pause/time semantics and plugin loading.
2. [Gazebo EntityComponentManager](#), entity/component access.
3. [Official ros_gz_bridge documentation](#), message bridge configuration.
4. [Gazebo GUI 8 plugins](#), native C++/QML widgets and plugin search paths.
5. Project Model 0-D v1.6 and Decisions 03/04 at the repository commit recorded in the delivery references. These provide the scoped geometric formulation; this guide adds engineering scenario choices.

14 Where to edit each part

`demo/demo_1_no_obstacle/` contains Demo 1's package, route input, expected milestones and media. `demo/demo_2_static_sphere/` contains the corresponding Demo 2 files. `common/src/um6p_demo_core/` contains `Motion.hh`, `Routes.hh`, the Gazebo system, native clearance panel and common bridge/launch configuration. `common/tests/` holds shared integration and geometry checks. Campus meshes exist only in the description package under `campus_environment/`.

To study or extend a route, read its two lines of x y z triplets, follow the generator into the SDF plugin fields, then follow Routes.hh into Motion. Rebuild after generation. Do not add a second copy of shared motion logic to a scenario package. Any change in routes, bounds, vehicle count or geometry requires new clearance tests; the current expected milestone files deliberately detect motion changes.

A Complete first-party source listings

These listings import the delivered files directly. Generated mesh coordinates are represented by the complete generator and separately hashed assets.

File 1

common/scripts/generate_demo_worlds.py

```
#!/usr/bin/env python3
"""Generate standalone demonstration worlds; never edit the paused world/assets."""
from pathlib import Path
import xml.etree.ElementTree as ET
root=Path(__file__).resolve().parents[2]
for wave in (1,2):
    slug='demo_1_no_obstacle' if wave==1 else 'demo_2_static_sphere'
    pkg=root/f'demo/{slug}/src/um6p_demo_{wave}'
    routes=(pkg/'config/routes.txt').read_text().splitlines()
    assert len(routes)==2
    for line in routes: assert len(line.split())%3==0 and all(__import__('math').isfinite(float(x))
        for x in line.split())
    tree=ET.parse(root/'campus_environment/src/um6p_campus_description/worlds/campus.sdf')
    world=tree.getroot().find('world'); world.set('name','um6p_demo')
    physics=ET.SubElement(world,'physics',name='kinematic_clock',type='ignored')
    ET.SubElement(physics,'max_step_size').text='0.02'
    ET.SubElement(physics,'real_time_factor').text='1'
    plugin=ET.SubElement(world,'plugin',filename='CampusKinematics',name='campus::Kinematics')
    ET.SubElement(plugin,'wave').text=str(wave)
    for i,line in enumerate(routes,1): ET.SubElement(plugin,f'route_{i}').text=line
    for inc in world.findall('include'):
        if inc.findtext('name').startswith('uav_'):
            ET.SubElement(inc,'static').text='false'
    pads=ET.SubElement(world,'model',name='landing_pads')
    ET.SubElement(pads,'static').text='true'; link=ET.SubElement(pads,'link',name='body')
    for i,x in enumerate((-3.6,3.6)):
        visual=ET.SubElement(link,'visual',name=f'pad_{i}')
        ET.SubElement(visual,'pose').text=f'{x} -40 .06 0 0 0'
        geom=ET.SubElement(visual,'geometry'); cyl=ET.SubElement(geom,'cylinder')
        ET.SubElement(cyl,'radius').text='1.2'; ET.SubElement(cyl,'length').text='.12'
        mat=ET.SubElement(visual,'material')
        for tag in ('ambient','diffuse'): ET.SubElement(mat,tag).text='.13 .15 .17 1'
    apron=ET.SubElement(world,'model',name='demo_apron')
    ET.SubElement(apron,'static').text='true'
    link=ET.SubElement(apron,'link',name='body')
    visual=ET.SubElement(link,'visual',name='paving')
    ET.SubElement(visual,'pose').text='0 -35 -.15 0 0 0'
    ET.SubElement(ET.SubElement(ET.SubElement(visual,'geometry'),'box'),'size').text='26 20 .3'
    mat=ET.SubElement(visual,'material')
    for field in ('ambient','diffuse'): ET.SubElement(mat,field).text='.60 .55 .44 1'
    if wave==2:
        ball=ET.SubElement(world,'model',name='static_sphere')
        ET.SubElement(ball,'static').text='true'
        ET.SubElement(ball,'pose').text='0 -22 2.4 0 0 0'
        link=ET.SubElement(ball,'link',name='body')
        for tag in ('visual','collision'):
            element=ET.SubElement(link,tag,name='sphere')
            ET.SubElement(ET.SubElement(ET.SubElement(element,'geometry'),'sphere'),'radius').text
            ='2.4'
            if tag=='visual':
                mat=ET.SubElement(element,'material')
                for field in ('ambient','diffuse'): ET.SubElement(mat,field).text='.65 .07 .06 1'
    ET.indent(tree); tree.write(pkg/'worlds/demo.sdf',encoding='utf-8',xml_declaration=True)
    print('Generated wave1.sdf and wave2.sdf; original campus world unchanged.')
```

File 2

common/src/um6p_demo_core/CMakeLists.txt

```
cmake_minimum_required(VERSION 3.16)
project(um6p_demo_core)
cmake_policy(SET CMP0100 NEW)
find_package(ament_cmake REQUIRED)
find_package(gz_sim_vendor REQUIRED)
find_package(gz-sim8 REQUIRED)
find_package(gz_plugin_vendor REQUIRED)
find_package(gz-plugin2 REQUIRED COMPONENTS register)
find_package(gz_gui_vendor REQUIRED)
find_package(gz-gui8 REQUIRED)
find_package(Qt5 REQUIRED COMPONENTS Core Quick QuickControls2)
set(CMAKE_AUTOMOC ON)
qt5_add_resources(panel_resources config/panel.qrc)
add_library(ClearancePanel SHARED src/ClearancePanel.cc include/ClearancePanel.hh ${panel_resources})
target_compile_features(ClearancePanel PUBLIC cxx_std_17)
target_include_directories(ClearancePanel PRIVATE include)
target_link_libraries(ClearancePanel gz-gui8::gz-gui8 gz-plugin2::gz-plugin2 Qt5::Core Qt5::Quick Qt5::QuickControls2)
install(TARGETS ClearancePanel DESTINATION lib)
add_library(CampusKinematics SHARED src/CampusKinematics.cc)
target_compile_features(CampusKinematics PUBLIC cxx_std_17)
target_include_directories(CampusKinematics PRIVATE include)
target_link_libraries(CampusKinematics gz-sim8::gz-sim8 gz-plugin2::gz-plugin2)
add_executable(motion_check tests/motion_check.cc)
target_compile_features(motion_check PUBLIC cxx_std_17)
target_include_directories(motion_check PRIVATE include)
install(TARGETS CampusKinematics DESTINATION lib)
install(TARGETS motion_check DESTINATION lib/${PROJECT_NAME})
install(DIRECTORY launch config DESTINATION share/${PROJECT_NAME})
add_executable(panel_check tests/panel_check.cc)
target_link_libraries(panel_check Qt5::Core Qt5::Quick)
install(TARGETS panel_check DESTINATION lib/${PROJECT_NAME})
ament_package()
```

File 3

common/src/um6p_demo_core/config/ClearancePanel.qml

```
import QtQuick 2.12
import QtQuick.Layouts 1.12

Rectangle {
    id: panel
    width: 360
    height: obstacle ? 334 : 176
    color: "#ed13232e"
    radius: 14
    property var sample: ({})
    property bool obstacle: sample.wave === 2
    property double updated: 0
    property bool fresh: false
    function accept() {
        try { sample = JSON.parse(ClearancePanel.state); updated = Date.now(); fresh = true; }
        catch (error) { fresh = false; }
    }
    Connections { target: ClearancePanel; onStateChanged: panel.accept() }
    Timer { interval: 250; running: true; repeat: true
        onTriggered: panel.fresh = Date.now() - panel.updated < 1500 }
}
```

```

function distance(index) { return sample.distances ? sample.distances[index] : 0; }
function minimum(index) { return index === 0 ? 3.2 : 4.2; }
function shade(index) {
  if (!fresh || !sample.distances) return "#9ba9b2";
  var margin = distance(index) - minimum(index);
  return margin < 0 ? "#ff646b" : margin < 0.8 ? "#ffbe59" : "#62e5ac";
}
function label(index) {
  if (!fresh || !sample.distances) return "NO LIVE DATA";
  var margin = distance(index) - minimum(index);
  return margin < 0 ? "VIOLATION" : margin < 0.8 ? "NEAR LIMIT" : "CLEAR";
}
Column {
  anchors.fill: parent; anchors.margins: 16; spacing: 9
  Row {
    spacing: 10
    Text { text: "LIVE CLEARANCE"; color: "#ffffff"; font.pixelSize: 17; font.bold: true }
    Text { text: panel.sample.paused ? "PAUSED" : panel.sample.status === "COMPLETE" ? "LANDED" :
      "LIVE"
      color: "#a9bac7"; font.pixelSize: 12; anchors.baseline: parent.children[0].baseline }
  }
  Text { text: "Wave " + (panel.sample.wave || "\u2013") + " \u00b7 " +
    (panel.sample.sim_time || 0).toFixed(1) + " s \u00b7 " +
    (panel.sample.phases ? panel.sample.phases.join(" / ") : "Waiting for scene")
    color: "#c5d3dc"; font.pixelSize: 12 }
  Repeater {
    model: panel.obstacle ? 3 : 1
    delegate: Rectangle {
      width: 328; height: 70; radius: 8; color: "#293c49"
      Rectangle { width: 4; height: 46; radius: 2; x: 10; y: 12; color: panel.shade(index) }
      Text { x: 24; y: 8; text: index === 0 ? "UAV 1 \u2194 UAV 2" : "UAV " + index + " \u2194
        SPHERE"
        color: "#e5edf3"; font.pixelSize: 13; font.bold: true }
      Text { x: 24; y: 28; text: panel.fresh && panel.sample.distances ? panel.distance(index).
        toFixed(2) + " m" : "\u2014"
        color: panel.shade(index); font.pixelSize: 27; font.bold: true }
      Text { x: 196; y: 12; text: panel.label(index); color: panel.shade(index); font.pixelSize:
        12; font.bold: true }
      Text { x: 196; y: 33; text: "min " + panel.minimum(index).toFixed(2) + " m"
        color: "#c5d3dc"; font.pixelSize: 12 }
      Text { x: 196; y: 49; text: "margin " + (panel.distance(index)-panel.minimum(index)).
        toFixed(2) + " m"
        color: "#c5d3dc"; font.pixelSize: 11 }
    }
  }
}
}
}

```

File 4

common/src/um6p_demo_core/config/bridge.yaml

```

- ros_topic_name: /campus/demo/state
  gz_topic_name: /campus/demo/state
  ros_type_name: std_msgs/msg/String
  gz_type_name: gz.msgs.StringMsg
  direction: GZ_TO_ROS
- ros_topic_name: /clock
  gz_topic_name: /world/um6p_demo/clock
  ros_type_name: rosgraph_msgs/msg/Clock
  gz_type_name: gz.msgs.Clock
  direction: GZ_TO_ROS

```

File 5

common/src/um6p_demo_core/config/gui.config

```
<?xml version="1.0"?>
<window><width>1600</width><height>1000</height>
  <menus><drawer default="false"/></menus>
  <dialog_on_exit>false</dialog_on_exit>
</window>
<plugin filename="MinimalScene" name="Campus | KINEMATIC demonstration">
  <gz-gui><title>Campus | KINEMATIC demonstration</title>
    <property type="bool" key="showTitleBar">false</property>
    <property type="string" key="state">docked</property></gz-gui>
  <engine>ogre2</engine><scene>scene</scene>
  <ambient_light>.65 .65 .65</ambient_light>
  <background_color>.55 .72 .86</background_color>
  <camera_pose>22 -44 24 0 .64 2.38</camera_pose>
</plugin>
<plugin filename="GzSceneManager" name="Scene manager">
  <gz-gui><property key="state" type="string">floating</property>
    <property key="showTitleBar" type="bool">false</property>
    <property key="width" type="double">5</property>
    <property key="height" type="double">5</property></gz-gui>
</plugin>
<plugin filename="InteractiveViewControl" name="Camera inspection">
  <gz-gui><property key="state" type="string">floating</property>
    <property key="showTitleBar" type="bool">false</property>
    <property key="width" type="double">5</property>
    <property key="height" type="double">5</property></gz-gui>
</plugin>
<plugin filename="Screenshot" name="Screenshot">
  <gz-gui><property key="state" type="string">floating</property>
    <property key="showTitleBar" type="bool">false</property>
    <property key="width" type="double">50</property>
    <property key="height" type="double">50</property></gz-gui>
</plugin>

<plugin filename="CameraTracking" name="Saved camera views">
  <gz-gui><property key="state" type="string">floating</property>
    <property key="showTitleBar" type="bool">false</property>
    <property key="width" type="double">5</property>
    <property key="height" type="double">5</property></gz-gui>
</plugin>

<plugin filename="ClearancePanel" name="Live clearance">
  <gz-gui>
    <title>Live clearance</title>
    <property type="bool" key="showTitleBar">false</property>
    <property type="string" key="state">floating</property>
    <property type="double" key="width">360</property>
    <property type="double" key="height">350</property>
    <property type="double" key="x">1220</property>
    <property type="double" key="y">16</property>
    <property type="double" key="z">10</property>
  </gz-gui>
</plugin>
```

File 6

common/src/um6p_demo_core/config/panel.qrc

```
<RCC><qresource prefix="ClearancePanel"><file>ClearancePanel.qml</file></qresource></RCC>
```

File 7

common/src/um6p_demo_core/include/ClearancePanel.hh

```
#pragma once
#include <gz/gui/Plugin.hh>
#include <gz/transport/Node.hh>
#include <gz/messages/stringmsg.pb.h>
#include <gz/messages/boolean.pb.h>
#include <QString>

class ClearancePanel: public gz::gui::Plugin {
    Q_OBJECT
    Q_PROPERTY(QString state READ State NOTIFY StateChanged)
public:
    QString State() const {return state;}
    void LoadConfig(const tinyxml2::XMLElement*) override;
signals:
    void StateChanged();
private:
    QString state="{}";
    gz::transport::Node node;
    void Receive(const gz::messages::StringMsg&);
    bool Capture(const gz::messages::StringMsg&,gz::messages::Boolean&);
};
```

File 8

common/src/um6p_demo_core/include/Motion.hh

```
#pragma once
// First-party deterministic kinematics. No optimizer or rigid-body dynamics.
#include <algorithm>
#include <array>
#include <cmath>
#include <stdexcept>
#include <string>
#include <vector>

namespace campus {
struct Vec {
    double x{}, y{}, z{};
    Vec operator+(Vec b) const {return {x+b.x,y+b.y,z+b.z};}
    Vec operator-(Vec b) const {return {x-b.x,y-b.y,z-b.z};}
    Vec operator*(double s) const {return {x*s,y*s,z*s};}
    double dot(Vec b) const {return x*b.x+y*b.y+z*b.z;}
    double norm() const {return std::sqrt(dot(*this));}
};
inline double segmentDistance(Vec a, Vec b) {
    const Vec d=b-a;
    const double t=d.dot(d)>0 ? std::clamp(-a.dot(d)/d.dot(d),0.0,1.0):0;
    return (a+d*t).norm();
}
inline Vec toward(Vec p, Vec target, double step) {
    Vec d=target-p; double n=d.norm();
    return n<=step ? target : p+d*(step/n);
}
constexpr double radius=1.2, pairGap=.8, obstacleGap=.6;
constexpr double pairRequired=2*radius+pairGap;
constexpr double ballRadius=2.4, ballRequired=ballRadius+radius+obstacleGap;
const Vec bodyOffset{0,0,.36}, ball{0,-22,2.4};
const std::array<Vec,2> starts{{{ -3.6,-8.2,.12},{3.6,-8.2,.12}}};
```

```

struct Motion {
    int wave;
    std::array<Vec,2> p=starts;
    std::array<std::vector<Vec>,2> route;
    std::array<size_t,2> next{{0,0}};
    double elapsed=0, minPair=1e9, minBall=1e9;
    size_t yieldTicks=0, rejectedTicks=0;
    bool failed=false, yielding=false;
    std::string reason;
    explicit Motion(int w, std::array<std::vector<Vec>,2> routes):wave(w),route(std::move(routes)) {
        if (w!=1 && w!=2) throw std::invalid_argument("wave must be 1 or 2");
        for(const auto& points:route) {
            if(points.empty()) throw std::invalid_argument("empty route");
            for(auto v:points) if(!std::isfinite(v.norm())) throw std::invalid_argument("invalid waypoint");
        }
    }
    bool done() const {return next[0]==route[0].size() && next[1]==route[1].size();}
    std::string phase(int i) const {
        if(failed) return "FAILED";
        if(done() || next[i]==route[i].size()) return "LANDED";
        if(elapsed<3) return "READY";
        if(next[i]==0) return "TAKEOFF";
        if(i==1 && yielding && next[i]>0) return "YIELD";
        if(next[i]+1==route[i].size()) return "LANDING";
        return wave==2 ? "BYPASS" : "TRAVEL";
    }
    bool admissible(const std::array<Vec,2>& q) const {
        if(segmentDistance(p[0]-p[1],q[0]-q[1])<pairRequired+1e-6) return false;
        for(int i=0;i<2;++i) {
            // Convex reserved corridor; endpoints imply the full segment stays inside.
            // Ground contact is checked by mesh z bounds, not the inflated sphere.
            for(Vec v:{p[i],q[i]})
                if(!std::isfinite(v.norm()) || v.x < -7 || v.x > 7 ||
                    v.y < -41 || v.y > -7 || v.z < .12-1e-9 || v.z > 2.8+1e-9) return false;
            if(wave==2 && segmentDistance(p[i]+bodyOffset-ball,q[i]+bodyOffset-ball)<ballRequired+1e-6)
                return false;
        }
        return true;
    }
    bool remainingLegsClear() const {
        // Predict constant-speed travel to each current waypoint, then hold there.
        // Arrival times split the prediction into synchronized linear intervals.
        std::array<Vec,2> goal=p;
        std::array<double,3> times{{0,0,0}};
        for(int i=0;i<2;++i) if(next[i]<route[i].size()) {
            goal[i]=route[i][next[i]];
            times[i+1]=(goal[i]-p[i]).norm()/.85;
        }
        std::sort(times.begin(),times.end());
        auto relative=[&](double t){
            return toward(p[0],goal[0],.85*t)-toward(p[1],goal[1],.85*t);};
        for(int k=1;k<3;++k)
            if(segmentDistance(relative(times[k-1]),relative(times[k]))<pairRequired+.32)
                return false;
        return true;
    }
    void advance(double dt) {
        if(!(dt>0 && dt<=.0200001)) throw std::invalid_argument("invalid time step");
        if(failed || done()) return;
        elapsed+=dt;
        if(elapsed<3) return;
        auto q=p;
        std::array<Vec,2> forecast=p;
        for(int i=0;i<2;++i) if(next[i]<route[i].size())

```

```

        forecast[i]=toward(p[i],route[i][next[i]],.85*1.0);
        if(yielding) yielding=!remainingLegsClear();
        else yielding=next[i]>0 && next[i]<route[i].size() &&
            segmentDistance(p[0]-p[1],forecast[0]-forecast[1])<pairRequired+.3;

        for(int i=0;i<2;++i) {
            if(next[i]==route[i].size()) continue;
            if(phase(i=="YIELD") {++yieldTicks; continue;}
            q[i]=toward(p[i],route[i][next[i]],.85*dt);
        }
        if(!admissible(q)) {q=p; ++rejectedTicks;}
        minPair=std::min(minPair,segmentDistance(p[0]-p[1],q[0]-q[1]));
        if(wave==2) for(int i=0;i<2;++i)
            minBall=std::min(minBall,segmentDistance(p[i]+bodyOffset-ball,q[i]+bodyOffset-ball));
        p=q;
        for(int i=0;i<2;++i)
            if(next[i]<route[i].size() && (p[i]-route[i][next[i]]).norm()<1e-9) ++next[i];
        if(elapsed>120 && !done()) {failed=true;reason="mission timeout / blocked path";}
    }
};
}

```

File 9

common/src/um6p_demo_core/include/Routes.hh

```

#pragma once
#include "Motion.hh"
#include <sstream>
namespace campus {
inline std::vector<Vec> parseRoute(const std::string& text) {
    std::istringstream input(text); std::vector<Vec> points; Vec v;
    while(input >> v.x) {
        if(!(input >> v.y >> v.z)) throw std::invalid_argument("waypoint needs x y z");
        points.push_back(v);
    }
    if(!input.eof() || points.empty()) throw std::invalid_argument("invalid route text");
    return points;
}
}

```

File 10

common/src/um6p_demo_core/launch/demo.launch.py

```

"""Separate explicitly enabled kinematic mode. Default starts paused."""
from pathlib import Path
import os
from ament_index_python.packages import get_package_share_directory, get_package_prefix
from launch import LaunchDescription
from launch.actions import DeclareLaunchArgument, ExecuteProcess, OpaqueFunction,
    RegisterEventHandler, EmitEvent
from launch.event_handlers import OnProcessExit
from launch.events import Shutdown
from launch.substitutions import LaunchConfiguration
from launch_ros.actions import Node

def start(context):
    demo=Path(get_package_share_directory('um6p_demo_core'))
    scenario=Path(get_package_share_directory(LaunchConfiguration('scenario_package').perform(
        context)))

```

```

desc=Path(get_package_share_directory('um6p_campus_description'))
bringup=Path(get_package_share_directory('um6p_campus_bringup'))
gui=LaunchConfiguration('gui').perform(context)=='true'
command=['gz','sim','-v','3',str(scenario/'worlds/demo.sdf')]
if gui: command+=['--gui-config',str(demo/'config/gui.config')]
else: command+=['-s']
gazebo=ExecuteProcess(cmd=command,output='screen',additional_env={
    'GZ_SIM_RESOURCE_PATH':str(desc/'models'),
    'GZ_SIM_SYSTEM_PLUGIN_PATH':str(Path(get_package_prefix('um6p_demo_core'))/'lib'),
    'GZ_GUI_PLUGIN_PATH':str(Path(get_package_prefix('um6p_demo_core'))/'lib'),
    'GZ_SIM_SERVER_CONFIG_PATH':str(bringup/'config/server.config')})
bridge=Node(package='ros_gz_bridge',executable='parameter_bridge',name='campus_demo_bridge',
    parameters=[{'config_file':str(demo/'config/bridge.yaml')}],output='screen',
    namespace=os.environ.get('CAMPUS_VERIFY_NAMESPACE',''),
    remappings=[('/campus/demo/state','/'+os.environ['CAMPUS_VERIFY_NAMESPACE']+'/state
'),
                ('/clock','/'+os.environ['CAMPUS_VERIFY_NAMESPACE']+'/clock')]
    if os.environ.get('CAMPUS_VERIFY_NAMESPACE') else [])
return [gazebo,bridge,RegisterEventHandler(OnProcessExit(target_action=gazebo,
    on_exit=[EmitEvent(event=Shutdown(reason='Gazebo exited'))])])]

def generate_launch_description():
    return LaunchDescription([
        DeclareLaunchArgument('scenario_package',choices=['um6p_demo_1','um6p_demo_2']),
        DeclareLaunchArgument('gui',default_value='true',choices=['true','false']),
        OpaqueFunction(function=start)])

```

File 11

common/src/um6p_demo_core/package.xml

```

<?xml version="1.0"?>
<package format="3">
  <name>um6p_demo_core</name><version>1.0.0</version>
  <description>Deterministic two-UAV kinematic showcases, no flight dynamics.</description>
  <maintainer email="ismo-idi@users.noreply.github.com">Ismael Assoumane Idi</maintainer>
  <license>UNLICENSED</license>
  <buildtool_depend>ament_cmake</buildtool_depend>
  <depend>gz_sim_vendor</depend><depend>gz_plugin_vendor</depend>
  <depend>gz_gui_vendor</depend><depend>qtbase5-dev</depend><depend>qtdeclarative5-dev</depend>
  <depend>qtquickcontrols2-5-dev</depend>
  <exec_depend>um6p_campus_description</exec_depend>
  <exec_depend>um6p_campus_bringup</exec_depend>
  <exec_depend>launch_ros</exec_depend><exec_depend>ament_index_python</exec_depend>
  <exec_depend>ros_gz_bridge</exec_depend><exec_depend>rosgraph_msgs</exec_depend>
  <exec_depend>std_msgs</exec_depend>
  <export><build_type>ament_cmake</build_type></export>
</package>

```

File 12

common/src/um6p_demo_core/src/CampusKinematics.cc

```

#include "Routes.hh"
#include <gz/sim/System.hh>
#include <gz/sim/EntityComponentManager.hh>
#include <gz/sim/components/Model.hh>
#include <gz/sim/components/Name.hh>
#include <gz/sim/components/Pose.hh>
#include <gz/plugin/Register.hh>
#include <gz/transport/Node.hh>

```

```

#include <gz/msgs/stringmsg.pb.h>
#include <cstdlib>
#include <fstream>
#include <iomanip>
#include <memory>
#include <sstream>

namespace campus {
class Kinematics: public gz::sim::System, public gz::sim::ISystemConfigure,
    public gz::sim::ISystemPreUpdate, public gz::sim::ISystemPostUpdate {
    std::unique_ptr<Motion> motion;
    std::array<gz::sim::Entity,2> entities{};
    gz::transport::Node node;
    gz::transport::Node::Publisher publisher;
    std::ofstream csv;
    uint64_t tick=0;
    bool bound=false;
public:
    void Configure(const gz::sim::Entity&,const std::shared_ptr<const sdf::Element>& sdf,
        gz::sim::EntityComponentManager&,gz::sim::EventManager&) override {
        motion=std::make_unique<Motion>(sdf->Get<int>("wave"),
            std::array<std::vector<Vec>,2>{{parseRoute(sdf->Get<std::string>("route_1")),
                parseRoute(sdf->Get<std::string>("route_2"))}}});
        publisher=node.Advertise<gz::msgs::StringMsg>("/campus/demo/state");
        const char* path=std::getenv("CAMPUS_TRACE");
        if(path) {csv.open(path); if(!csv) throw std::runtime_error("Cannot open CAMPUS_TRACE");
            csv<<"tick,sim_time,pause,x1,y1,z1,x2,y2,z2,phase1,phase2,status\n"<<std::setprecision(17);}
    }
    void PreUpdate(const gz::sim::UpdateInfo& info,gz::sim::EntityComponentManager& ecm) override {
        if(!bound) {
            for(int i=0;i<2;++i)entities[i]=ecm.EntityByComponents(
                gz::sim::components::Model(),gz::sim::components::Name("uav_"+std::to_string(i+1)));
            bound=entities[0] && entities[1];
        }
        if(!bound || info.paused) return;
        double dt=std::chrono::duration<double>(info.dt).count();
        if(dt<=0) return;
        if(std::abs(dt-.02)>1e-8) {motion->failed=true;motion->reason="step must equal 0.02s";return;}
        // Reject external pose changes instead of silently overwriting them.
        for(int i=0;i<2;++i) {
            auto pose=ecm.Component<gz::sim::components::Pose>(entities[i])>Data();
            Vec actual{pose.Pos().X(),pose.Pos().Y(),pose.Pos().Z()};
            if((actual-motion->p[i]).norm(>1e-8) {
                motion->failed=true;motion->reason="unexpected pose modification";return;
            }
        }
        motion->advance(dt);++tick;
        for(int i=0;i<2;++i) {
            Vec p=motion->p[i];
            ecm.SetComponentData<gz::sim::components::Pose>(entities[i],
                gz::math::Pose3d(p.x,p.y,p.z,0,0,i==0?.3:-.3));
            ecm.SetChanged(entities[i],gz::sim::components::Pose::typeId,
                gz::sim::ComponentState::PeriodicChange);
        }
    }
    void PostUpdate(const gz::sim::UpdateInfo& info,const gz::sim::EntityComponentManager& ecm)
        override {
        if(!bound) return;
        const double time=std::chrono::duration<double>(info.simTime).count();
        const std::string status=motion->failed?"FAILED":motion->done()?"COMPLETE":"RUNNING";
        std::ostringstream s;s<<std::setprecision(17);
        s<<("{\"tick\": \"<<tick<<\", \"sim_time\": \"<<time<<\", \"paused\": \"<<(info.paused?"true":"false")
            <<\", \"wave\": \"<<motion->wave<<\", \"status\": \"<<status<<\", \"poses\": [\";
        if(csv)csv<<tick<<\", \"<<time<<\", \"<<info.paused;
        for(int i=0;i<2;++i) {

```

```

    auto p=ecm.Component<gz::sim::components::Pose>(entities[i])->Data().Pos();
    if(i)s<<" ";s<<"["<<p.X()<<" "<<p.Y()<<" "<<p.Z()<<"]";
    if(csv)csv<<" "<<p.X()<<" "<<p.Y()<<" "<<p.Z();
}

s<<"],\"distances\":["<<(motion->p[0]-motion->p[1]).norm()<<" "<<
    (motion->p[0]+bodyOffset-ball).norm()<<" "<<(motion->p[1]+bodyOffset-ball).norm();
s<<"],\"phases\":["<<"<<motion->phase(0)<<"\"<<"<<motion->phase(1)
    <<"\"<<,"<<"yield_ticks\"<<:"<<motion->yieldTicks<<,"<<"rejected_ticks\"<<:"<<motion->rejectedTicks
    <<,"<<"reason\"<<:"<<"<<motion->reason<<"\"<<"}";
if(csv) {csv<<" "<<motion->phase(0)<<" "<<motion->phase(1)<<" "<<status<<"\n";
    if(motion->done() || motion->failed)csv.flush();}
// Publisher is telemetry only. The simulator's update thread owns both UAV poses.
if(tick%5==0 || info.paused || motion->done() || motion->failed) {
    gz::msgs::StringMsg msg;msg.set_data(s.str());publisher.Publish(msg);
}
}
};
}

GZ_ADD_PLUGIN(campus::Kinematics,gz::sim::System,campus::Kinematics::ISystemConfigure,
    campus::Kinematics::ISystemPreUpdate,campus::Kinematics::ISystemPostUpdate)

```

File 13

```
common/src/um6p_demo_core/src/ClearancePanel.cc
```

```
#include "ClearancePanel.hh"
#include <gz/plugin/Register.hh>
#include <QQuickItem>
#include <QQuickWindow>
#include <QImage>
#include <QJsonDocument>
#include <QJsonObject>

void ClearancePanel::LoadConfig(const tinyxml2::XMLElement*) {
    node.Subscribe("/campus/demo/state",&ClearancePanel::Receive,this);
    node.Advertise("/campus/demo/hud_capture",&ClearancePanel::Capture,this);
}

void ClearancePanel::Receive(const gz::msgs::StringMsg& msg) {
    const QString next=QString::fromStdString(msg.data());
    // Transport callback runs off the GUI thread; all QML state changes are queued.
    QMetaObject::invokeMethod(this,[this,next]){
        state=next;
        auto data=QJsonDocument::fromJson(next.toUtf8()).object();
        if(auto card=CardItem()) {
            card->setProperty("height",data["wave"].toInt()==2?334:176);
            if(auto parent=card->parentItem())
                card->setX(qMax(16.0,parent->width()-376.0));
        }
        emit StateChanged();
    },Qt::QueuedConnection);
}

bool ClearancePanel::Capture(const gz::msgs::StringMsg& request,gz::msgs::Boolean& reply) {
    bool saved=false;
    const QString path=QString::fromStdString(request.data());
    QMetaObject::invokeMethod(this,[this,path,&saved]{
        auto item=PluginItem();
        if(item && item->window())saved=item->window()->grabWindow().save(path,"PNG");
    },Qt::BlockingQueuedConnection);
    reply.set_data(saved);return saved;
}

GZ_ADD_PLUGIN(ClearancePanel,gz::gui::Plugin)
```

File 14

common/src/um6p_demo_core/tests/motion_check.cc

```
#include "Routes.hh"
#include <fstream>
#include <iostream>
#include <iomanip>
int main(int argc, char**argv) {
    using namespace campus;
    auto require=[](bool b){if(!b) throw std::runtime_error("core check failed");};
    require(segmentDistance({-2,0,0},{2,0,0})==0);
    require(segmentDistance({3,0,0},{3,0,0})==3);
    require(segmentDistance({3,0,0},{4,0,0})==3);
    require(argc==3);
    std::array<std::vector<Vec>,2>,2> routes;
    for(int w=0;w<2;++w) {
        std::ifstream file(argv[w+1]); require(bool(file));
        for(int i=0;i<2;++i) {std::string line; require(bool(std::getline(file,line))); routes[w][i]=
            parseRoute(line);}
    }
    Motion crossing(1,routes[0]);
    crossing.p={Vec{-3.6,-10,2.8},Vec{3.6,-10,2.8}};
    require(!crossing.admissible({crossing.p[1],crossing.p[0]}));
    Motion throughBall(2,routes[1]);
    throughBall.p={Vec{-5.6,-22,2.8},Vec{6,-7.5,2.8}};
    require(!throughBall.admissible({Vec{5.6,-22,2.8},throughBall.p[1]}));
    Motion blocked(1,routes[0]);
    blocked.route[0]={Vec{100,-8.2,2.8}};
    for(int k=0;k<6100 && !blocked.failed;++k)blocked.advance(.02);
    require(blocked.failed && !blocked.done());
    for(int wave:{1,2}) {
        Motion a(wave,routes[wave-1]),b(wave,routes[wave-1]);
        int episodes=0;bool wasYielding=false;
        for(int k=0;k<6100 && !a.done() && !a.failed;++k){
            a.advance(.02);b.advance(.02);
            if(a.yielding && !wasYielding)++episodes;
            wasYielding=a.yielding;
            require((a.p[0]-b.p[0]).norm()==0 && (a.p[1]-b.p[1]).norm()==0);
            if(a.next[0]==0 && a.next[1]==0)require(a.p[0].z==a.p[1].z);
        }
        require(episodes==1);
        require(a.done() && !a.failed && a.rejectedTicks==0);
        require((a.p[0]-b.p[0]).norm()==0 && a.elapsed==b.elapsed);
        require(a.minPair>=pairRequired && (wave==1 || a.minBall>=ballRequired));
        require(wave!=1 || a.yieldTicks>0);
        std::cout<<"wave="<<wave<<" complete t="<<a.elapsed<<" min_pair="<<a.minPair
            <<" min_ball="<<a.minBall<<" yield_ticks="<<a.yieldTicks<<"\\n";
    }
}
```

File 15

common/src/um6p_demo_core/tests/panel_check.cc

```
// Test the actual QML color functions with synthetic data, without Gazebo motion.
#include <QGuiApplication>
#include <QFile>
#include <QQmlEngine>
#include <QQmlComponent>
#include <QQmlContext>
#include <QVariant>
#include <QJsonDocument>
```

```

#include <QJsonObject>
#include <QJsonArray>
#include <iostream>
#include <stdexcept>
class MockSample: public QObject {
    Q_OBJECT
    Q_PROPERTY(QString state READ State NOTIFY StateChanged)
public:
    QString value="{}";
    QString State() const {return value;}
signals:
    void StateChanged();
};
int main(int argc,char**argv) {
    QGuiApplication app(argc,argv); QQmlEngine engine;
    auto require=[](bool ok){if(!ok)throw std::runtime_error("HUD check failed");};
    require(argc==2);
    MockSample sample;
    engine.rootContext()->setContextProperty("ClearancePanel",&sample);
    QFile file(argv[1]);require(file.open(QIODevice::ReadOnly));
    QQmlComponent component(&engine);
    component.setData(file.readAll(),QUrl("qrc:/panel-test.qml"));
    std::unique_ptr<QObject> panel(component.create());
    if(!panel){std::cerr<<component.errorString().toStdString();return 1;}
    int cases=0;
    for(int wave:{1,2})for(int index=0;index<(wave==1?1:3);++index) {
        auto call=[&](const char*method){QVariant out;
            require(QMetaObject::invokeMethod(panel.get(),method,Q_RETURN_ARG(QVariant,out),Q_ARG(QVariant
            ,index)));
            return out.toString();};
        for(double margin:{-.01,0.,.79,.81}) {
            QJsonArray distances{7.2,8.,8.};distances[index]=(index==0?3.2:4.2)+margin;
            QJsonObject data{"wave",wave},{"distances",distances},{"paused",true},
                {"sim_time",12.},{"phases",QJsonArray{"TRAVEL","YIELD"}}};
            sample.value=QString::fromUtf8(QJsonDocument(data).toJson());
            emit sample.StateChanged();
            app.processEvents();
            require(call("label")== (margin<0?"VIOLATION":margin<.8?"NEAR LIMIT":"CLEAR"));
            require(call("shade")== (margin<0?"#ff646b":margin<.8?"#ffbe59":"#62e5ac"));
            require(panel->property("height").toInt()==(wave==2?334:176));++cases;
        }
        panel->setProperty("fresh",false);
        require(call("label")=="NO LIVE DATA" && call("shade")=="#9ba9b2");++cases;
    }
    std::cout<<"{"status\":"PASS\","cases\":"<<cases
        <<","scope\":"real QML; synthetic data only; no unsafe simulation\"}\n";
}

#include "panel_check.moc"

```

File 16

common/tests/check_cli.py

```

#!/usr/bin/env python3
"""Invalid commands must explain the error, never silently start or exit."""
import subprocess
from pathlib import Path
root=Path(__file__).resolve().parents[2]
for args in ([],['3'],['1','guibash','scripts/demo_control.sh','start'],['2','bad']):
    p=subprocess.run(['bash',str(root/'scripts/launch_demo.sh'),*args],capture_output=True,text=
        True,timeout=5)
    assert p.returncode==2 and 'Terminal A:' in p.stderr and 'Terminal B:' in p.stderr

```

```
print('PASS: four invalid launch commands return actionable two-terminal instructions')
```

File 17

common/tests/check_demo_geometry.py

```
#!/usr/bin/env python3
"""Conservative mesh-bound checks for the fixed showcase, independent of Motion.hh."""
import csv,json,math,sys
from pathlib import Path
import xml.etree.ElementTree as ET
ROOT=Path(__file__).resolve().parents[2]
PKG=ROOT/'campus_environment/src/um6p_campus_description'

def vertices(path):
    v=[];f=[]
    for line in path.read_text().splitlines():
        if line.startswith('v '):v.append(tuple(map(float,line.split()[1:])))
        elif line.startswith('f '):f.append([int(x.split('/')[0])-1 for x in line.split()[1:]])
    return v,f

def point_box(p,lo,hi):
    return sum(max(lo[i]-p[i],0,p[i]-hi[i])**2 for i in range(3))

def segment_box(a,b,lo,hi):
    d=[b[i]-a[i] for i in range(3)];cuts={0.,1.}
    for i in range(3):
        if d[i]:
            for bound in (lo[i],hi[i]):
                t=(bound-a[i])/d[i]
                if 0<t<1:cuts.add(t)
    cuts=sorted(cuts);best=min(point_box(a,lo,hi),point_box(b,lo,hi))
    for left,right in zip(cuts,cuts[1:]):
        mid=(left+right)/2;num=den=0
        for i in range(3):
            value=a[i]+mid*d[i]
            bound=lo[i] if value<lo[i] else hi[i] if value>hi[i] else None
            if bound is not None:num+=d[i]*(a[i]-bound);den+=d[i]**2
        t=max(left,min(right,-num/den)) if den else mid
        best=min(best,point_box([a[i]+t*d[i] for i in range(3)],lo,hi))
    return math.sqrt(best)

def check(trace):
    radii={}
    for name in ('uav_1','uav_2'):
        vs=[v for path in (PKG/'models'/name/'meshes').glob('*.obj') for v in vertices(path)[0]]
        radii[name]=max(math.dist(v,(0,0,.36)) for v in vs)
        assert radii[name]<1.2
        assert min(v[2] for v in vs)>=.005-1e-9
    for wave in (1,2):
        demo=ET.parse(ROOT/'demo'/'demo_1_no_obstacle' if wave==1 else 'demo_2_static_sphere')/f'
src/um6p_demo_{wave}/worlds/demo.sdf').getroot().find('world')
        plugins=[p.attrib['filename'] for p in demo.findall('plugin')]
        assert plugins==['gz-sim-scene-broadcaster-system','CampusKinematics'],plugins
        assert float(demo.findtext('physics/max_step_size'))==.02
        apron=demo.find("model[@name='demo_apron']/link/visual")
        assert list(map(float,apron.findtext('pose').split()))==[0,-35,-.15,0,0,0]
        assert list(map(float,apron.findtext('geometry/box/size').split()))==[26,20,.3]
        # Apron covers the corridor beyond original courtyard edge y=-25.
        assert -13 < -8.2 and 13 > 8.2 and -45 < -42.2
        pads=demo.findall("model[@name='landing_pads']/link/visual")
        assert sorted(tuple(map(float,v.findtext('pose').split()[:3])) for v in pads)
==[(-3.6,-40,.06),(3.6,-40,.06)]
```

```

assert all(float(v.findtext('geometry/cylinder/length'))==.12 for v in pads)
sphere=demo.find("model[@name='static_sphere']")
assert (sphere is not None)==(wave==2)
if sphere is not None:
    assert sphere.findtext('static')==True
    assert list(map(float,sphere.findtext('pose').split()[1:3]))==[0,-22,2.4]
    assert float(sphere.findtext('link/visual/geometry/sphere/radius'))==2.4
boxes=[];total=0
world=ET.parse(PKG/'worlds/campus.sdf').getroot().find('world')
for inc in world.findall('include'):
    name=inc.findtext('name')
    if name in ('courtyard','uav_1','uav_2'):continue
    pose=list(map(float,inc.findtext('pose').split()));assert pose[3]==[0,0,0]
    for path in (PKG/'models'/name/'meshes').glob('*.obj'):
        vs,faces=vertices(path)
        for face in faces:
            total+=1
            triangle=[[vs[j][i]+pose[i] for i in range(3)] for j in face]
            lo=[min(v[i] for v in triangle) for i in range(3)]
            hi=[max(v[i] for v in triangle) for i in range(3)]
            # Entire flight center region inflated by 1.2m; discard irrelevant triangles.
            if hi[0]<-8.2 or lo[0]>8.2 or hi[1]<-42.2 or lo[1]>-5.8 or lo[2]>4.36:continue
            boxes.append((lo,hi,name))
rows=list(csv.DictReader(Path(trace).open()));minimum=1e9;last=None
for row in rows:
    ps=[[float(row[f'{axis}{i}']) for axis in 'xyz'] for i in (1,2)]
    for v in ps:v[2]+=.36
    if last is not None and ps!=last:
        for i in range(2):
            for lo,hi,name in boxes:minimum=min(minimum,segment_box(last[i],ps[i],lo,hi))
        last=ps
assert total>0
assert minimum>1.2,minimum
return {'status':'PASS','radius_bound':1.2,'measured_radii':radii,
        'scenery_triangles_inspected':total,'near_corridor_triangle_boxes':len(boxes),
        'min_body_center_to_scenery_box':minimum if boxes else None,
        'corridor_clear_by_broadphase':not boxes,
        'ground_check':'minimum vertex z = model z + .005; z>=.12, pads top .12; paving top 0',
        'scope':'fixed geometry; sphere/inter-UAV checked separately; conservative triangle-AABB distance'}

if __name__=='__main__':print(json.dumps(check(sys.argv[1]),indent=2))

```

File 18

common/tests/verify_demo.py

```

#!/usr/bin/env python3
"""Bounded integration harness for the explicitly authorized moving showcase."""
import argparse, csv, json, math, os, signal, subprocess, time, uuid
from pathlib import Path
import rclpy
from std_msgs.msg import String
from roscpp_msgs.msg import Clock
from gz.transport13 import Node
from gz.msgs10.empty_pb2 import Empty
from gz.msgs10.scene_pb2 import Scene
from gz.msgs10.pose_v_pb2 import Pose_V
from google.protobuf import text_format

ROOT=Path(__file__).resolve().parents[2]
p=argparse.ArgumentParser();p.add_argument('wave',type=int,choices=[1,2])
p.add_argument('--gui',action='store_true');p.add_argument('--output',required=True)

```

```

a=p.parse_args();out=Path(a.output).resolve();out.mkdir(parents=True,exist_ok=False)
namespace='campus_verify_'+uuid.uuid4().hex
os.environ.update(GZ_PARTITION=namespace, CAMPUS_VERIFY_NAMESPACE=namespace, ROS_DOMAIN_ID='85',
    ROS_AUTOMATIC_DISCOVERY_RANGE='LOCALHOST',CAMPUS_TRACE=str(out/'trace.csv'),
    ROS_LOG_DIR=str(ROOT/'log/demo-verification'))
rclpy.init();node=rclpy.create_node('demo_verifier',namespace='/' + namespace);states=[];clocks=[];gz
    =Node()
render_poses={}
def render_update(message):
    for pose in message.pose:
        render_poses[pose.id]=[pose.position.x,pose.position.y,pose.position.z]
gz.subscribe(Pose_V,'/world/um6p_demo/dynamic_pose/info',render_update)
node.create_subscription(String,'/' + namespace + '/state',lambda m:states.append(json.loads(m.data))
    ,100)
node.create_subscription(Clock,'/' + namespace + '/clock',lambda m:clocks.append(m.clock.sec+m.clock.
    nanosec*1e-9),100)
result={'status':'FAIL','wave':a.wave,'gui':a.gui,'ros_namespace':namespace,'gz_partition':
    namespace}
result['isolation']='unique Gazebo partition and ROS topics/node namespace'

def wait_for(predicate,seconds=40):
    end=time.monotonic()+seconds
    while time.monotonic()<end:
        if proc.poll() is not None:raise RuntimeError('launch exited early')
        rclpy.spin_once(node,timeout_sec=.05)
        if states and states[-1]['status']== 'FAILED':raise RuntimeError(states[-1]['reason'])
        if predicate():return
    raise TimeoutError('condition timed out')

def service(name,reqtype,request):
    r=subprocess.run(['gz','service','-s',name,'--reqtype',reqtype,
        '--reptype','gz.msgs.Boolean','--timeout','5000','--req',request],
        text=True,capture_output=True,timeout=8)
    assert r.returncode==0 and 'data: true' in r.stdout,(name,r.stdout,r.stderr)

def control(paused):
    service('/world/um6p_demo/control','gz.msgs.WorldControl','pause: '+str(paused).lower())

def pause_and_confirm():
    before=len(states)
    minimum_tick=states[-1]['tick']
    control(True)
    wait_for(lambda:len(states)>before and states[-1]['paused']
        and states[-1]['tick']>=minimum_tick)
    return states[-1]

def scene_info():
    # Use bounded CLI exchange: the Python Scene request intermittently failed in headless mode.
    end=time.monotonic()+20
    while time.monotonic()<end:
        reply=subprocess.run(['gz','service','-s','/world/um6p_demo/scene/info',
            '--reqtype','gz.msgs.Empty','--reptype','gz.msgs.Scene','--timeout','3000',
            '--req',''],capture_output=True,text=True,timeout=5)
        try:
            scene=text_format.Parse(reply.stdout,Scene())
            if len(scene.model)==(10 if a.wave==2 else 9):return scene
        except text_format.ParseError:pass
        rclpy.spin_once(node,timeout_sec=.1)
    raise TimeoutError('scene service did not become ready')

def screenshot(filename="hud.png"):
    # Camera-only reset, using the same installed configuration as the launch.
    import re
    from ament_index_python.packages import get_package_share_directory
    from gz.msgs10.gui_camera_pb2 import GUICamera

```

```

from gz.msgs10.stringmsg_pb2 import StringMsg
cfg=Path(get_package_share_directory('um6p_demo_core'))/'config/gui.config'
x,y,z,roll,pitch,yaw=map(float,re.search(r'<camera_pose>(.*?)</camera_pose>',cfg.read_text())
[1].split())
c=GUICamera();c.pose.position.x=x;c.pose.position.y=y;c.pose.position.z=z
c.pose.orientation.x=-math.sin(yaw/2)*math.sin(pitch/2)
c.pose.orientation.y=math.cos(yaw/2)*math.sin(pitch/2)
c.pose.orientation.z=math.sin(yaw/2)*math.cos(pitch/2)
c.pose.orientation.w=math.cos(yaw/2)*math.cos(pitch/2)
service('/gui/move_to/pose','gz.msgs.GUICamera',str(c));time.sleep(2)
service('/campus/demo/hud_capture','gz.msgs.StringMsg',str(StringMsg(data=str(out/filename))))
assert (out/filename).is_file()

log=(out/'launch.log').open('w')
proc=subprocess.Popen([str(ROOT/'scripts/launch_demo.sh'),str(a.wave),
'gui' if a.gui else 'headless'],stdout=log,stderr=subprocess.STDOUT,start_new_session=True)
try:
    wait_for(lambda: len(states)>=3)
    assert all(s['paused'] and s['sim_time']==0 and s['wave']==a.wave for s in states)
    scene=scene_info()
    names=sorted(m.name for m in scene.model)
    expected=['demo_apron','campus_buildings','courtyard','landing_pads','landscaping','pergola',
'uav_1','uav_2','um6p_landmark']+(['static_sphere'] if a.wave==2 else [])
    assert names==sorted(expected)
    assert [n for n in names if n.startswith('uav_')]==['uav_1','uav_2']
    assert ('static_sphere' in names)==(a.wave==2)
    result['initial_entities']=names
    ids={m.name:m.id for m in scene.model}
    control(False)
    if a.wave==2:
        wait_for(lambda:min(states[-1]['distances'][1:])<4.5,50)
        pause_and_confirm()
        result['near_obstacle_state']=states[-1]
        if a.gui:screenshot('obstacle.png')
        control(False)
    wait_for(lambda:states[-1]['phases'][1]=='YIELD' and states[-1]['distances'][0]<4.0,80)
    pause_and_confirm()
    hold=states[-1];mark=len(states);end=time.monotonic()+1.2
    while time.monotonic(<end:rclpy.spin_once(node,timeout_sec=.05)
    assert len(states)>mark
    assert all(s['poses']==hold['poses'] and s['sim_time']==hold['sim_time'] for s in states[mark:])

    result['pause_check']='PASS';result['midflight_state']=hold
    if a.gui:screenshot()
    control(False);wait_for(lambda:states[-1]['status']=='COMPLETE',70)
    result['completed_state']=states[-1]
    assert states[-1]['poses']==[[3.6,-40,.12],[-3.6,-40,.12]]
    assert states[-1]['yield_ticks']>0 and states[-1]['rejected_ticks']==0
    for state in states:
        assert state['wave']==a.wave, 'Foreign scenario telemetry received'
        assert abs(math.dist(*state['poses'])-state['distances'][0])<1e-8
        if a.wave==2:
            for i,pose in enumerate(state['poses']):
                center=[pose[0],pose[1],pose[2]+.36]
                assert abs(math.dist(center,[0,-22,2.4])-state['distances'][i+1])<1e-8
    result['telemetry_distances']='PASS'

    assert clocks and max(clocks)>10,'No advancing ROS clock exchange'
    assert sorted(name for name,ns in node.get_node_names_and_namespaces()
        if ns=='/namespace')==['campus_demo_bridge','demo_verifier']
    scene=scene_info()
    # Scene/info describes the scene graph; use dynamic render poses for motion.
    wait_for(lambda:all(ids[f'uav_{i+1}'] in render_poses and
        math.dist(render_poses[ids[f'uav_{i+1}']],states[-1]['poses'][i])<1e-8
        for i in range(2)),5)

```

```

    actual={f'uav_{i+1}':render_poses[ids[f'uav_{i+1}']] for i in range(2)}
    result['final_render_poses']=actual
    for i in range(2):
        assert math.dist(actual[f'uav_{i+1}'],states[-1]['poses'][i])<1e-8
    (out/'final-scene.txt').write_text(str(scene))
    control(True)
    result['ros_clock_max']=max(clocks);result['ros_messages']=len(states)
finally:
    if proc.poll() is None:proc.send_signal(signal.SIGINT)
    try:proc.wait(timeout=15)
    except subprocess.TimeoutExpired:os.killpg(proc.pid,signal.SIGTERM);proc.wait(timeout=5)
    log.close();result['exit_code']=proc.returncode
    (out/'received-states.json').write_text(json.dumps(states)+'\n')
    (out/'result.json').write_text(json.dumps(result,indent=2)+'\n')
    node.destroy_node();rclpy.shutdown()

# Independent calculation from the per-update ECM trace, not planner margins.
def sub(a,b):return [x-y for x,y in zip(a,b)]
def norm(a):return math.sqrt(sum(x*x for x in a))
def closest(a,b):
    d=sub(b,a);den=sum(x*x for x in d)
    t=min(1,max(0,-sum(x*y for x,y in zip(a,d))/den)) if den else 0
    return norm([x+t*y for x,y in zip(a,d)])
rows=list(csv.DictReader((out/'trace.csv').open()));minimum_pair=1e9;minimum_ball=1e9
previous=None;unique=[]
for row in rows:
    q=[[float(row[f'{axis}-{i}']) for axis in 'xyz'] for i in (1,2)]
    assert all(math.isfinite(v) for p in q for v in p)
    if previous is not None:
        minimum_pair=min(minimum_pair,closest(sub(previous[0],previous[1]),sub(q[0],q[1])))
        for i in range(2):
            assert math.dist(previous[i],q[i])<=.85*.02+1e-8
            if a.wave==2:
                center=[0,-22,2.04] # sphere relative to UAV model origin; body center +.36z
                minimum_ball=min(minimum_ball,closest(sub(previous[i],center),sub(q[i],center)))
        for v in q:assert -7<=v[0]<=7 and -41<=v[1]<=-7 and .12-1e-9<=v[2]<=2.8+1e-9
    previous=q
    if not unique or int(row['tick'])!=unique[-1][0]:unique.append((int(row['tick']),q))
takeoff=[next(t for t,q in unique if q[i][2]>.12+1e-9) for i in range(2)]
assert takeoff[0]==takeoff[1]
yield_rows=[row for row in rows if row['phase2']=='YIELD']
assert yield_rows
first_yield=int(yield_rows[0]['tick']);last_yield=int(yield_rows[-1]['tick'])
assert any(int(row['tick'])>last_yield and row['phase2'] in ('TRAVEL','BYPASS') for row in rows)
scenario=ROOT/'demo/'('demo_1_no_obstacle' if a.wave==1 else 'demo_2_static_sphere')
expected=json.loads((scenario/'tests/expected.json').read_text())
assert takeoff[0]==expected['takeoff_tick']
assert int(rows[-1]['tick'])>=expected['completion_tick']
assert result['completed_state']['yield_ticks']==expected['yield_ticks']
result['simultaneous_takeoff_tick']=takeoff[0]
result['yield_interval_ticks']=[first_yield,last_yield]
unique_rows={int(row['tick']):row for row in rows}
assert all(unique_rows[t]['phase2']=='YIELD' for t in range(first_yield,last_yield+1))
result['single_continuous_yield']='PASS'
assert minimum_pair>=3.2+1e-6
if a.wave==2:assert minimum_ball>=4.2+1e-6
assert rows[-1]['status']=='COMPLETE'
assert proc.returncode==0
text=(out/'launch.log').read_text()
assert not any(x in text for x in ['[Err]', 'process has died', 'Missing material', 'Unable to find
    uri'])
from check_demo_geometry import check
result['geometry_check']=check(out/'trace.csv')
result.update(status='PASS',trace_rows=len(rows),min_pair=minimum_pair,
    min_ball=minimum_ball if a.wave==2 else None)

```

```
(out/'result.json').write_text(json.dumps(result,indent=2)+'\n')
print(json.dumps(result,indent=2))
```

File 19

demo/demo_1_no_obstacle/src/um6p_demo_1/CMakeLists.txt

```
cmake_minimum_required(VERSION 3.16)
project(um6p_demo_1)
find_package(ament_cmake REQUIRED)
install(DIRECTORY worlds config launch DESTINATION share/${PROJECT_NAME})
ament_package()
```

File 20

demo/demo_1_no_obstacle/src/um6p_demo_1/config/routes.txt

```
-3.6 -8.2 2.8 3.6 -40 2.8 3.6 -40 0.12
3.6 -8.2 2.8 -3.6 -40 2.8 -3.6 -40 0.12
```

File 21

demo/demo_1_no_obstacle/src/um6p_demo_1/launch/demo.launch.py

```
"""Demo 1; explicitly starts paused."""
from pathlib import Path
from ament_index_python.packages import get_package_share_directory
from launch import LaunchDescription
from launch.actions import DeclareLaunchArgument, IncludeLaunchDescription
from launch.launch_description_sources import PythonLaunchDescriptionSource
from launch.substitutions import LaunchConfiguration

def generate_launch_description():
    shared=Path(get_package_share_directory('um6p_demo_core'))/'launch/demo.launch.py'
    return LaunchDescription([
        DeclareLaunchArgument('gui',default_value='true',choices=['true','false']),
        IncludeLaunchDescription(PythonLaunchDescriptionSource(str(shared)),
            launch_arguments={'scenario_package':'um6p_demo_1',
                'gui':LaunchConfiguration('gui')}.items()))])
```

File 22

demo/demo_1_no_obstacle/src/um6p_demo_1/package.xml

```
<?xml version="1.0"?>
<package format="3">
  <name>um6p_demo_1</name><version>1.0.0</version>
  <description>Deterministic campus demo 1: crossing.</description>
  <maintainer email="ismo-idi@users.noreply.github.com">Ismael Assoumane Idi</maintainer>
  <license>UNLICENSED</license>
  <buildtool_depend>ament_cmake</buildtool_depend>
  <exec_depend>um6p_demo_core</exec_depend>
  <exec_depend>ament_index_python</exec_depend>
  <exec_depend>launch</exec_depend><exec_depend>launch_ros</exec_depend>
  <export><build_type>ament_cmake</build_type></export>
</package>
```

File 23

demo/demo_1_no_obstacle/src/um6p_demo_1/worlds/demo.sdf

```
<?xml version='1.0' encoding='utf-8'?>
<sdf version="1.10">
  <world name="um6p_demo">
    <plugin filename="gz-sim-scene-broadcaster-system" name="gz::sim::systems::SceneBroadcaster" />
    <scene>
      <ambient>.65 .65 .65 1</ambient>
      <background>.55 .72 .86 1</background>
      <shadows>true</shadows>
    </scene>
    <light name="warm_sun" type="directional">
      <pose>0 0 20 0 0 0</pose>
      <diffuse>.95 .86 .72 1</diffuse>
      <specular>.2 .2 .2 1</specular>
      <direction>-.45 .5 -.85</direction>
      <cast_shadows>true</cast_shadows>
    </light>
    <include>
      <uri>model://courtyard</uri>
      <name>courtyard</name>
      <pose>0 0 0 0 0 0</pose>
    </include>
    <include>
      <uri>model://pergola</uri>
      <name>pergola</name>
      <pose>0 0 0 0 0 0</pose>
    </include>
    <include>
      <uri>model://campus_buildings</uri>
      <name>campus_buildings</name>
      <pose>0 12 0 0 0 0</pose>
    </include>
    <include>
      <uri>model://um6p_landmark</uri>
      <name>um6p_landmark</name>
      <pose>-7 0 0 0 0 0</pose>
    </include>
    <include>
      <uri>model://landscaping</uri>
      <name>landscaping</name>
      <pose>0 0 0 0 0 0</pose>
    </include>
    <include>
      <uri>model://uav_1</uri>
      <name>uav_1</name>
      <pose>-3.6 -8.2 0.12 0 0 0.3</pose>
      <static>>false</static>
    </include>
    <include>
      <uri>model://uav_2</uri>
      <name>uav_2</name>
      <pose>3.6 -8.2 0.12 0 0 -0.3</pose>
      <static>>false</static>
    </include>
    <physics name="kinematic_clock" type="ignored">
      <max_step_size>0.02</max_step_size>
      <real_time_factor>1</real_time_factor>
    </physics>
    <plugin filename="CampusKinematics" name="campus::Kinematics">
      <wave>1</wave>
      <route_1>-3.6 -8.2 2.8 3.6 -40 2.8 3.6 -40 0.12</route_1>
      <route_2>3.6 -8.2 2.8 -3.6 -40 2.8 -3.6 -40 0.12</route_2>
```

```

</plugin>
<model name="landing_pads">
  <static>true</static>
  <link name="body">
    <visual name="pad_0">
      <pose>-3.6 -40 .06 0 0 0</pose>
      <geometry>
        <cylinder>
          <radius>1.2</radius>
          <length>.12</length>
        </cylinder>
      </geometry>
      <material>
        <ambient>.13 .15 .17 1</ambient>
        <diffuse>.13 .15 .17 1</diffuse>
      </material>
    </visual>
    <visual name="pad_1">
      <pose>3.6 -40 .06 0 0 0</pose>
      <geometry>
        <cylinder>
          <radius>1.2</radius>
          <length>.12</length>
        </cylinder>
      </geometry>
      <material>
        <ambient>.13 .15 .17 1</ambient>
        <diffuse>.13 .15 .17 1</diffuse>
      </material>
    </visual>
  </link>
</model>
<model name="demo_apron">
  <static>true</static>
  <link name="body">
    <visual name="paving">
      <pose>0 -35 -.15 0 0 0</pose>
      <geometry>
        <box>
          <size>26 20 .3</size>
        </box>
      </geometry>
      <material>
        <ambient>.60 .55 .44 1</ambient>
        <diffuse>.60 .55 .44 1</diffuse>
      </material>
    </visual>
  </link>
</model>
</world>
</sdf>

```

File 24

demo/demo_1_no_obstacle/tests/expected.json

```

{
  "wave": 1,
  "takeoff_tick": 150,
  "completion_tick": 2596,
  "yield_ticks": 213,
  "goals": [
    [

```

```

    3.6,
    -40,
    0.12
  ],
  [
    -3.6,
    -40,
    0.12
  ]
]
}

```

File 25

demo/demo_2_static_sphere/src/um6p_demo_2/CMakeLists.txt

```

cmake_minimum_required(VERSION 3.16)
project(um6p_demo_2)
find_package(ament_cmake REQUIRED)
install(DIRECTORY worlds config launch DESTINATION share/${PROJECT_NAME})
ament_package()

```

File 26

demo/demo_2_static_sphere/src/um6p_demo_2/config/routes.txt

```

-3.6 -8.2 2.8 -4.3 -19 2.8 -4.3 -27 2.8 3.6 -40 2.8 3.6 -40 0.12
3.6 -8.2 2.8 4.3 -19 2.8 4.3 -27 2.8 -3.6 -40 2.8 -3.6 -40 0.12

```

File 27

demo/demo_2_static_sphere/src/um6p_demo_2/launch/demo.launch.py

```

"""Demo 2; explicitly starts paused."""
from pathlib import Path
from ament_index_python.packages import get_package_share_directory
from launch import LaunchDescription
from launch.actions import DeclareLaunchArgument, IncludeLaunchDescription
from launch.launch_description_sources import PythonLaunchDescriptionSource
from launch.substitutions import LaunchConfiguration

def generate_launch_description():
    shared=Path(get_package_share_directory('um6p_demo_core'))/'launch/demo.launch.py'
    return LaunchDescription([
        DeclareLaunchArgument('gui', default_value='true', choices=['true', 'false']),
        IncludeLaunchDescription(PythonLaunchDescriptionSource(str(shared)),
                                launch_arguments={'scenario_package': 'um6p_demo_2',
                                                    'gui': LaunchConfiguration('gui')}.items()))
    ])

```

File 28

demo/demo_2_static_sphere/src/um6p_demo_2/package.xml

```

<?xml version="1.0"?>
<package format="3">
  <name>um6p_demo_2</name><version>1.0.0</version>
  <description>Deterministic campus demo 2: static sphere bypass.</description>
  <maintainer email="ismo-idi@users.noreply.github.com">Ismael Assoumane Idi</maintainer>

```

```

<license>UNLICENSED</license>
<buildtool_depend>ament_cmake</buildtool_depend>
<exec_depend>um6p_demo_core</exec_depend>
<exec_depend>ament_index_python</exec_depend>
<exec_depend>launch</exec_depend><exec_depend>launch_ros</exec_depend>
<export><build_type>ament_cmake</build_type></export>
</package>

```

File 29

demo/demo_2_static_sphere/src/um6p_demo_2/worlds/demo.sdf

```

<?xml version='1.0' encoding='utf-8'?>
<sdf version="1.10">
  <world name="um6p_demo">
    <plugin filename="gz-sim-scene-broadcaster-system" name="gz::sim::systems::SceneBroadcaster" />
    <scene>
      <ambient>.65 .65 .65 1</ambient>
      <background>.55 .72 .86 1</background>
      <shadows>true</shadows>
    </scene>
    <light name="warm_sun" type="directional">
      <pose>0 0 20 0 0 0</pose>
      <diffuse>.95 .86 .72 1</diffuse>
      <specular>.2 .2 .2 1</specular>
      <direction>-.45 .5 -.85</direction>
      <cast_shadows>true</cast_shadows>
    </light>
    <include>
      <uri>model://courtyard</uri>
      <name>courtyard</name>
      <pose>0 0 0 0 0 0</pose>
    </include>
    <include>
      <uri>model://pergola</uri>
      <name>pergola</name>
      <pose>0 0 0 0 0 0</pose>
    </include>
    <include>
      <uri>model://campus_buildings</uri>
      <name>campus_buildings</name>
      <pose>0 12 0 0 0 0</pose>
    </include>
    <include>
      <uri>model://um6p_landmark</uri>
      <name>um6p_landmark</name>
      <pose>-7 0 0 0 0 0</pose>
    </include>
    <include>
      <uri>model://landscaping</uri>
      <name>landscaping</name>
      <pose>0 0 0 0 0 0</pose>
    </include>
    <include>
      <uri>model://uav_1</uri>
      <name>uav_1</name>
      <pose>-3.6 -8.2 0.12 0 0 0.3</pose>
      <static>false</static>
    </include>
    <include>
      <uri>model://uav_2</uri>
      <name>uav_2</name>
      <pose>3.6 -8.2 0.12 0 0 -0.3</pose>
    </include>
  </world>
</sdf>

```

```

    <static>false</static>
  </include>
  <physics name="kinematic_clock" type="ignored">
    <max_step_size>0.02</max_step_size>
    <real_time_factor>1</real_time_factor>
  </physics>
  <plugin filename="CampusKinematics" name="campus::Kinematics">
    <wave>2</wave>
    <route_1>-3.6 -8.2 2.8 -4.3 -19 2.8 -4.3 -27 2.8 3.6 -40 2.8 3.6 -40 0.12</route_1>
    <route_2>3.6 -8.2 2.8 4.3 -19 2.8 4.3 -27 2.8 -3.6 -40 2.8 -3.6 -40 0.12</route_2>
  </plugin>
  <model name="landing_pads">
    <static>true</static>
    <link name="body">
      <visual name="pad_0">
        <pose>-3.6 -40 .06 0 0 0</pose>
        <geometry>
          <cylinder>
            <radius>1.2</radius>
            <length>.12</length>
          </cylinder>
        </geometry>
        <material>
          <ambient>.13 .15 .17 1</ambient>
          <diffuse>.13 .15 .17 1</diffuse>
        </material>
      </visual>
      <visual name="pad_1">
        <pose>3.6 -40 .06 0 0 0</pose>
        <geometry>
          <cylinder>
            <radius>1.2</radius>
            <length>.12</length>
          </cylinder>
        </geometry>
        <material>
          <ambient>.13 .15 .17 1</ambient>
          <diffuse>.13 .15 .17 1</diffuse>
        </material>
      </visual>
    </link>
  </model>
  <model name="demo_apron">
    <static>true</static>
    <link name="body">
      <visual name="paving">
        <pose>0 -35 -.15 0 0 0</pose>
        <geometry>
          <box>
            <size>26 20 .3</size>
          </box>
        </geometry>
        <material>
          <ambient>.60 .55 .44 1</ambient>
          <diffuse>.60 .55 .44 1</diffuse>
        </material>
      </visual>
    </link>
  </model>
  <model name="static_sphere">
    <static>true</static>
    <pose>0 -22 2.4 0 0 0</pose>
    <link name="body">
      <visual name="sphere">
        <geometry>

```

```
<sphere>
  <radius>2.4</radius>
</sphere>
</geometry>
<material>
  <ambient>.65 .07 .06 1</ambient>
  <diffuse>.65 .07 .06 1</diffuse>
</material>
</visual>
<collision name="sphere">
  <geometry>
    <sphere>
      <radius>2.4</radius>
    </sphere>
  </geometry>
</collision>
</link>
</model>
</world>
</sdf>
```

File 30

demo/demo_2_static_sphere/tests/expected.json

```
{
  "wave": 2,
  "takeoff_tick": 150,
  "completion_tick": 2711,
  "yield_ticks": 243,
  "goals": [
    [
      3.6,
      -40,
      0.12
    ],
    [
      -3.6,
      -40,
      0.12
    ]
  ]
}
```

File 31

references/demo-packages.txt

```
# Additional/direct C++ development dependencies; existing stack already supplies them.
ros-jazzy-gz-sim-vendor
ros-jazzy-gz-plugin-vendor
ros-jazzy-ros-gz-bridge
ros-jazzy-rosgraph-msgs
ros-jazzy-std-msgs
build-essential
cmake
ros-jazzy-gz-gui-vendor
qtbase5-dev
qtdeclarative5-dev
qtquickcontrols2-5-dev
qml-module-qtquick-controls2
```

File 32

scripts/build.sh

```
#!/usr/bin/env bash
set -eo pipefail
ROOT="$(cd -- "$(dirname -- "${BASH_SOURCE[0]}")/.." && pwd)"
source /opt/ros/jazzy/setup.bash
cd "$ROOT"
colcon --log-base "$ROOT/log" build --base-paths "$ROOT/campus_environment/src" "$ROOT/common/src" \
    "$ROOT/demo/demo_1_no_obstacle/src" "$ROOT/demo/demo_2_static_sphere/src" \
    --build-base "$ROOT/build" --install-base "$ROOT/install" \
    --event-handlers console_direct+
```

File 33

scripts/build_kinematic_tutorial.sh

```
#!/usr/bin/env bash
exec bash "$(dirname -- "${BASH_SOURCE[0]}")/build_tutorials.sh" demos
```

File 34

scripts/build_tutorials.sh

```
#!/usr/bin/env bash
set -eo pipefail
ROOT="$(cd -- "$(dirname -- "${BASH_SOURCE[0]}")/.." && pwd)"
MODE="${1:-all}"
case "$MODE" in all|campus|demos) ;; *) echo 'Usage: build_tutorials.sh [all|campus|demos]' >&2;
    exit 2 ;; esac
python3 "$ROOT/scripts/tutorial_listings.py"
python3 "$ROOT/scripts/kinematic_listings.py"
for kind in campus_environment kinematic_demos; do
    [[ "$MODE" != campus || "$kind" == campus_environment ]] || continue
    [[ "$MODE" != demos || "$kind" == kinematic_demos ]] || continue
    cd "$ROOT/tutorial/source/$kind"
    if command -v latexmk >/dev/null; then
        latexmk -pdf -interaction=nonstopmode -halt-on-error tutorial.tex
    elif command -v tectonic >/dev/null; then
        tectonic --keep-logs tutorial.tex
    else
        echo 'Install optional documentation tools or provide Tectonic on PATH.' >&2; exit 2
    fi
    if [[ "$kind" == campus_environment ]]; then
        NAME=01_UM6P_Campus_Environment_Setup_and_Coding.pdf
    else
        NAME=02_Two_UAV_Kinematic_Demos_Coding_and_Verification.pdf
    fi
    cp tutorial.pdf "$ROOT/tutorial/$NAME"
    pdftitle "$ROOT/tutorial/$NAME"
done
```

File 35

scripts/demo_control.sh

```
#!/usr/bin/env bash
# Only controls the separately named demonstration world; no arbitrary world name.
```

```

set -eo pipefail
source /opt/ros/jazzy/setup.bash
export GZ_PARTITION="${GZ_PARTITION:-campus_demo}"
case "${1:-}" in start|resume) REQUEST='pause: false' ;; pause) REQUEST='pause: true' ;;
*) echo 'Usage: demo_control.sh start|pause|resume' >&2; exit 2 ;; esac
gz service -s /world/um6p_demo/control --reqtype gz.msgs.WorldControl \
--reptype gz.msgs.Boolean --timeout 5000 --req "$REQUEST"

```

File 36

scripts/kinematic_listings.py

```

#!/usr/bin/env python3
"""Import complete maintained source files into LaTeX without copying their contents."""
from pathlib import Path
r=Path(__file__).resolve().parents[1]
files=[]
for folder in ['common/src', 'common/scripts', 'common/tests', 'demo/demo_1_no_obstacle/src', 'demo/
demo_1_no_obstacle/tests', 'demo/demo_2_static_sphere/src', 'demo/demo_2_static_sphere/tests']:
    files += [p for p in (r/folder).rglob('*') if p.is_file() and '__pycache__' not in p.parts]
files += [r/'scripts'/n for n in ['build.sh', 'setup.sh', 'setup_demo.sh', 'launch_demo.sh',
demo_control.sh', 'verify_demo.sh', 'verify_unit.sh', 'reproduce_clean.sh', 'build_tutorials.sh',
build_kinematic_tutorial.sh', 'kinematic_listings.py']]
files += [r/'references/demo-packages.txt']
text='\section{Complete first-party source listings}\n'
text+='These listings import the delivered files directly. Generated mesh coordinates are
represented by the complete generator and separately hashed assets.\par\n'
for i,p in enumerate(sorted(set(files)),1):
    rel=p.relative_to(r).as_posix()
    text+='\subsection*{File '+str(i)+'}\n\n\noindent{\small\path{'+rel+'}}\par\n'
    text+='\lstinputlisting{../../'+rel+'}\n'
(r/'tutorial/source/kinematic_demos/listings.tex').write_text(text)
print('Imported',len(set(files)), 'source files for kinematic_demos')

```

File 37

scripts/launch_demo.sh

```

#!/usr/bin/env bash
set -eo pipefail
ROOT="$(cd -- "$(dirname -- "${BASH_SOURCE[0]}")/.." && pwd)"
usage() {
    echo 'Usage: bash scripts/launch_demo.sh 1|2 [gui|headless]' >&2
    echo 'Terminal A: bash scripts/launch_demo.sh 1 gui (or 2 gui)' >&2
    echo 'Wait for Gazebo. Terminal B: bash scripts/demo_control.sh start' >&2
    echo 'Enter each command separately; do not join the two terminal commands.' >&2
}
if [[ $# -lt 1 || $# -gt 2 || ! "$1" =~ ^[12]$ ]]; then usage; exit 2; fi
WAVE="$1"
case "${2:-gui}" in gui) GUI=true ;; headless) GUI=false ;; *) usage; exit 2 ;; esac
[[ -f "$ROOT/install/setup.bash" ]] || { echo 'Build first: bash scripts/build.sh' >&2; exit 2; }
source /opt/ros/jazzy/setup.bash
source "$ROOT/install/setup.bash"
export ROS_DOMAIN_ID="${ROS_DOMAIN_ID:-85}"
export ROS_AUTOMATIC_DISCOVERY_RANGE=LOCALHOST
export GZ_PARTITION="${GZ_PARTITION:-campus_demo}"
export ROS_LOG_DIR="$ROOT/log/demo"
export GZ_HOMEDIR="$ROOT/.runtime/gz-demo"
mkdir -p "$ROS_LOG_DIR" "$GZ_HOMEDIR"
echo "Kinematic wave $WAVE starts PAUSED. Use demo_control.sh start in another terminal."
exec ros2 launch "um6p_demo_$WAVE" demo.launch.py gui:="$GUI"

```

File 38

scripts/reproduce_clean.sh

```
#!/usr/bin/env bash
# Fresh build and bounded tests. Never overwrite a previous destination.
set -eo pipefail
ROOT="$(cd -- "${dirname -- "${BASH_SOURCE[0]}"}/.." && pwd)"
TARGET="${1:?Pass a new target directory (spaces supported)}"
[[ ! -e "$TARGET" ]] || { echo 'Target already exists; choose a fresh directory.' >&2; exit 2; }
TARGET="$(realpath -m -- "$TARGET")"
case "$TARGET/" in "$ROOT/*") echo 'Target must be outside source folder.' >&2; exit 2 ;; esac
mkdir -p "$TARGET"
for item in campus_environment common demo scripts references docs .vscode .gitignore; do
    cp -a -- "$ROOT/$item" "$TARGET/"
done
mkdir -p "$TARGET/evidence"
bash "$TARGET/scripts/build.sh" > "$TARGET/evidence/build.log" 2>&1
bash "$TARGET/scripts/verify_unit.sh" > "$TARGET/evidence/unit.log" 2>&1
bash "$TARGET/scripts/verify.sh" headless > "$TARGET/evidence/verification.log" 2>&1
for wave in 1 2; do
    bash "$TARGET/scripts/verify_demo.sh" "$wave" headless "$TARGET/evidence/wave$wave" > "$TARGET/
    evidence/wave$wave.log" 2>&1
done
printf 'Fresh build, paused campus and both kinematic demos PASS: %s\n' "$TARGET"
```

File 39

scripts/setup.sh

```
#!/usr/bin/env bash
# One dependency plan for campus and both demos; installs only with explicit option.
exec bash "$(dirname -- "${BASH_SOURCE[0]}")/setup_demo.sh" "$@"
```

File 40

scripts/setup_demo.sh

```
#!/usr/bin/env bash
# Additive dependency check; no installation without explicit --install-missing.
set -eo pipefail
ROOT="$(cd -- "${dirname -- "${BASH_SOURCE[0]}"}/.." && pwd)"
MODE="${1:---plan}"
case "$MODE" in --plan|--install-missing) ;; *) echo 'Usage: setup_demo.sh [--plan|--install-
missing]' >&2; exit 2 ;; esac
bash "$ROOT/scripts/setup_native.sh" "$MODE"
MISSING=()
while IFS= read -r package; do
    [[ -z "$package" || "$package" == \#* ]] && continue
    if [[ "$(dpkg-query -W -f='${db:Status-Status}' "$package" 2>/dev/null || true)" != installed ]];
    then MISSING+=("$package"); fi
done < "$ROOT/references/demo-packages.txt"
if (( ${#MISSING[@]} == 0 )); then echo 'All demo packages installed; no changes.'; exit 0; fi
printf 'Missing demo package: %s\n' "${MISSING[@]}"
if [[ "$MODE" == --install-missing ]]; then
    sudo apt-get update
    sudo apt-get install --no-upgrade "${MISSING[@]}"
else
    echo 'Review these packages; --install-missing installs only missing dependencies.'
fi
```

File 41

scripts/verify_demo.sh

```
#!/usr/bin/env bash
set -eo pipefail
ROOT="$(cd -- "$(dirname -- "${BASH_SOURCE[0]}")/.." && pwd)"
source /opt/ros/jazzy/setup.bash
source "$ROOT/install/setup.bash"
WAVE="${1:?Usage: verify_demo.sh 1|2 [gui|headless] [new-output-folder]}"
EXTRA=(); case "${2:-headless}" in gui) EXTRA=(--gui) ;; headless) ;; *) exit 2 ;; esac
OUTPUT="${3:-$ROOT/evidence/local-demo-$WAVE-$(date +%Y%m%d-%H%M%S)}"
timeout --signal=INT --kill-after=20s 150s python3 "$ROOT/common/tests/verify_demo.py" "$WAVE" \
    "${EXTRA[@]}" --output "$OUTPUT"
```

File 42

scripts/verify_unit.sh

```
#!/usr/bin/env bash
# Geometry/math and actual QML tests without advancing Gazebo.
set -eo pipefail
ROOT="$(cd -- "$(dirname -- "${BASH_SOURCE[0]}")/.." && pwd)"
source /opt/ros/jazzy/setup.bash
source "$ROOT/install/setup.bash"
python3 "$ROOT/common/tests/check_cli.py"
"$ROOT/install/um6p_demo_core/lib/um6p_demo_core/motion_check" \
    "$ROOT/demo/demo_1_no_obstacle/src/um6p_demo_1/config/routes.txt" \
    "$ROOT/demo/demo_2_static_sphere/src/um6p_demo_2/config/routes.txt"
QT_QPA_PLATFORM=offscreen "$ROOT/install/um6p_demo_core/lib/um6p_demo_core/panel_check" \
    "$ROOT/install/um6p_demo_core/share/um6p_demo_core/config/ClearancePanel.qml"
```